

机器学习导论

第10讲上：深度学习入门



人工智能系

晁飞副教授

fchao@xmu.edu.cn



廈門大學
XIAMEN UNIVERSITY

概述

图像识别目标

图像识别挑战

图像识别基本框架

图像识别理想目标

让计算机像人一样理解图像



我们看到的

0	3	2	5	4	7	6	9	8
3	0	1	2	3	4	5	6	7
2	1	0	3	2	5	4	7	6
5	2	3	0	1	2	3	4	5
4	3	2	1	0	3	2	5	4
7	4	5	2	3	0	1	2	3
6	5	4	3	2	1	0	3	2
9	6	7	4	5	2	3	0	1
8	7	6	5	4	3	2	1	0

计算机看到的

图像识别实际目标



让计算机将语义概念相似的图像划分为同一类别

Manuel Lagunas & Elena Garces / Transfer Learning for Illustration Classification

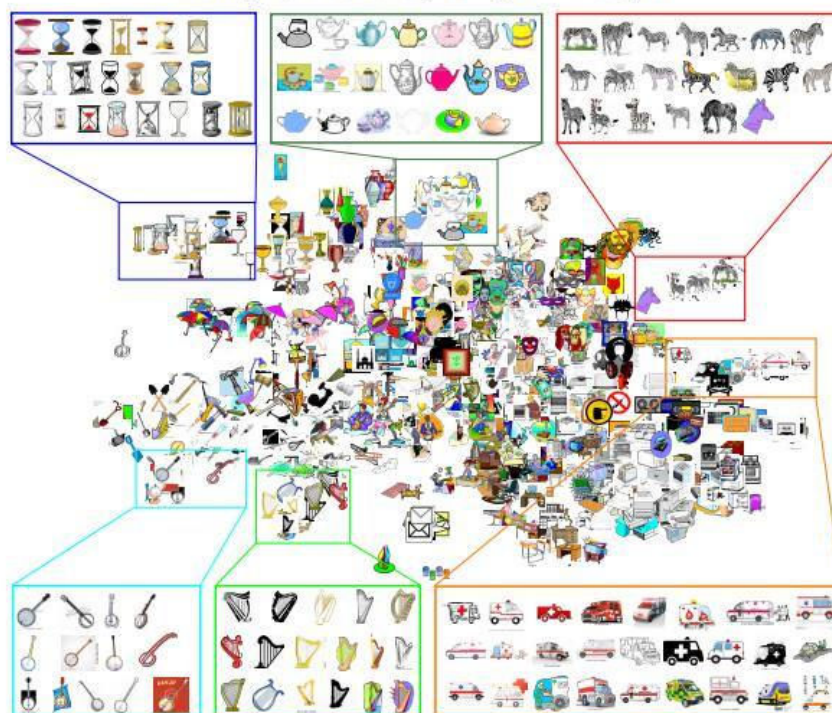


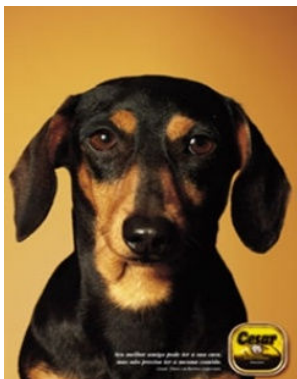
Figure 6: t-SNE algorithm on the image descriptors of the optimized network. The boxes show groups of images of the same class that have been grouped together after applying the algorithm. This shows us that the network is able to understand the low-level image characteristics and, consequently, that these image descriptors can be classified with a support vector machine obtaining great results.

图像识别面临挑战

语义鸿沟(Semantic Gap)现象

◆ **Semantic Gap:** the gap between **low-level visual features** and **high-level concepts** (图像的底层视觉特性和高层语义概念之间的鸿沟)

➤ 例如：相似的视觉特性(color, texture, shape, ...) ,不同的语义概念



图像识别面临挑战

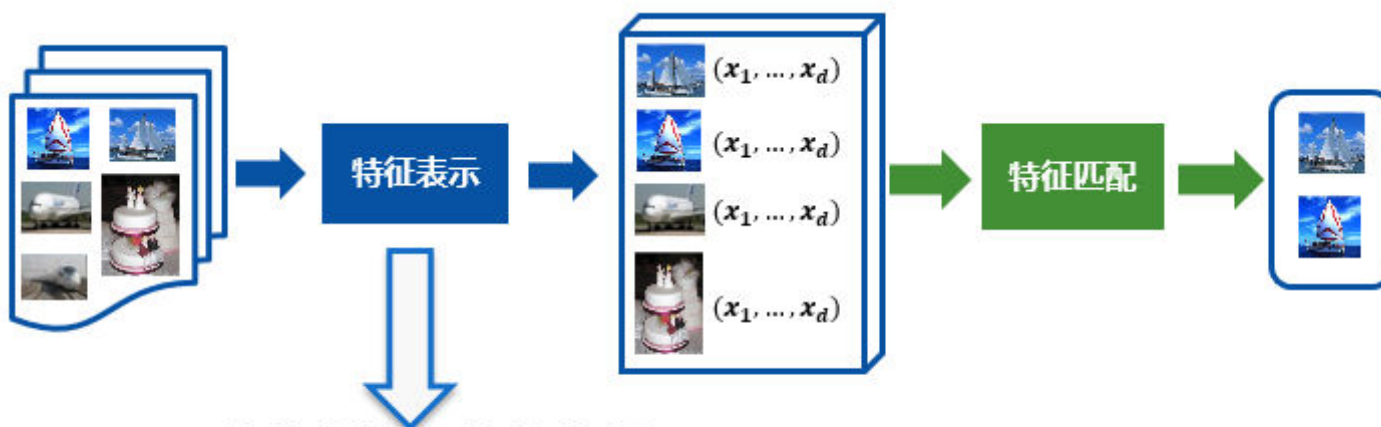
语义鸿沟(Semantic Gap)现象

➤ 又例如：不相似的视觉特性,相同的语义概念



图像识别基本框架

测量空间 $\longrightarrow$ 特征空间 $\longrightarrow$ 类别空间



传统方法：设计特征
深度学习：学习特征

图像识别基本框架（场景识别、目标识别、人脸识别...）

传统图像识别技术

早期图像识别技术(1990-2003)

中期图像识别技术(2003-2012)

深度学习与图像识别

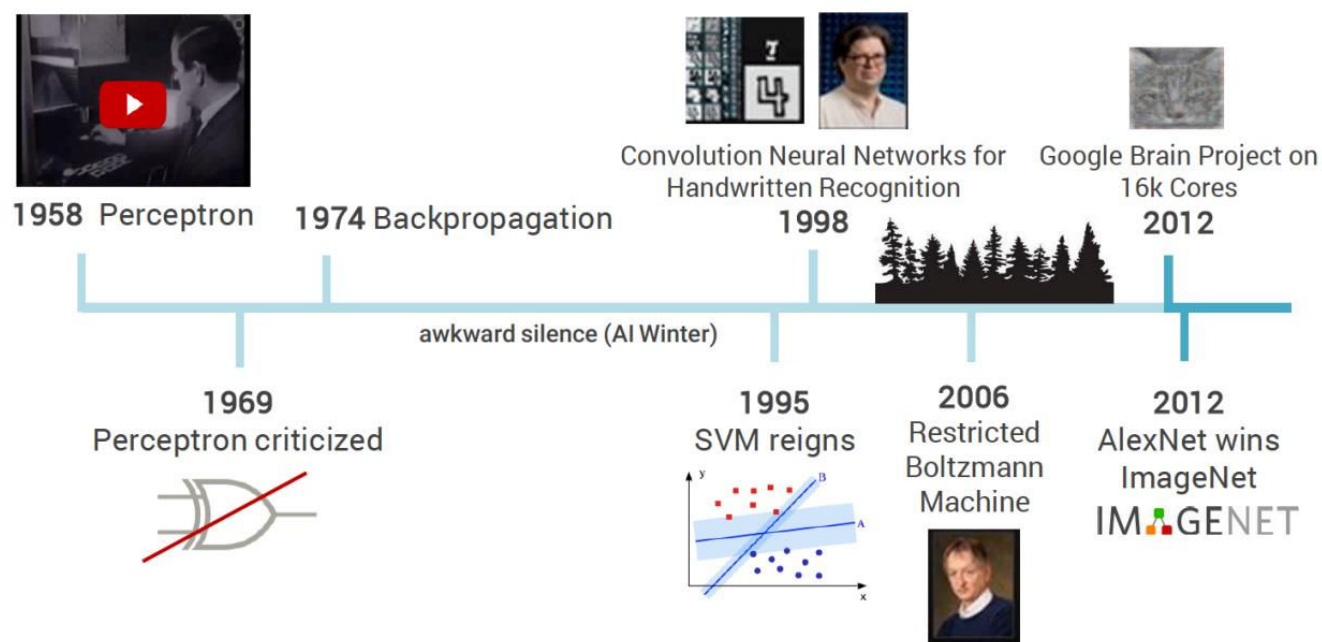
深度学习发展历程

为什么使用深度学习

如何使用深度学习

深度学习与图像识别

深度学习起源与发展



深度学习与图像识别

深度学习在图像领域的应用



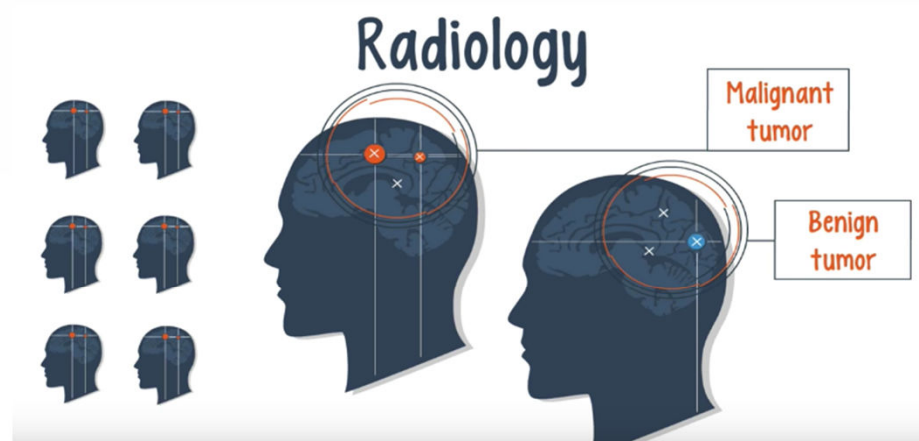
Predicted Tags

no person table elegant
indoors luxury furniture
fashion decoration tableware
party

Similar Images



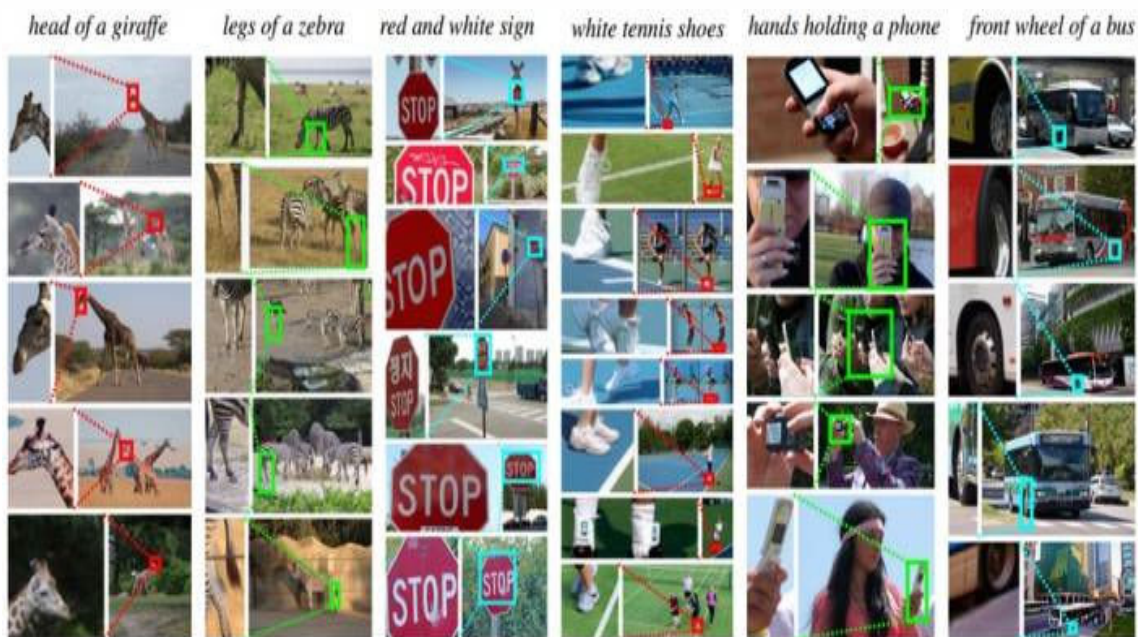
图片检索



识别异常的肿瘤或者癌细胞

深度学习与图像识别

深度学习在图像领域的应用



图片描述



图片着色

深度学习与图像识别

深度学习发展历程

为什么使用深度学习

如何使用深度学习

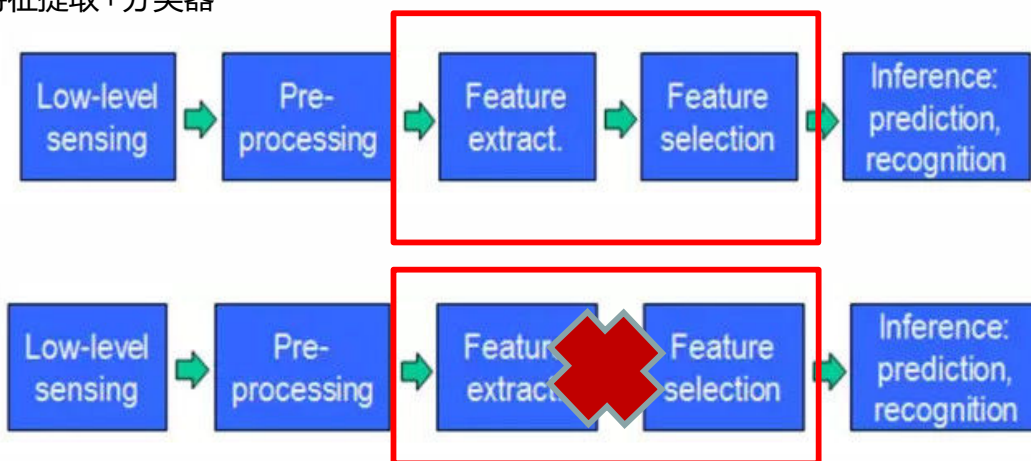
图像识别为什么使用深度学习

测量空间 $\longrightarrow$ 特征空间 $\longrightarrow$ 类别空间

核心任务

传统方法

- 人工特征提取+分类器



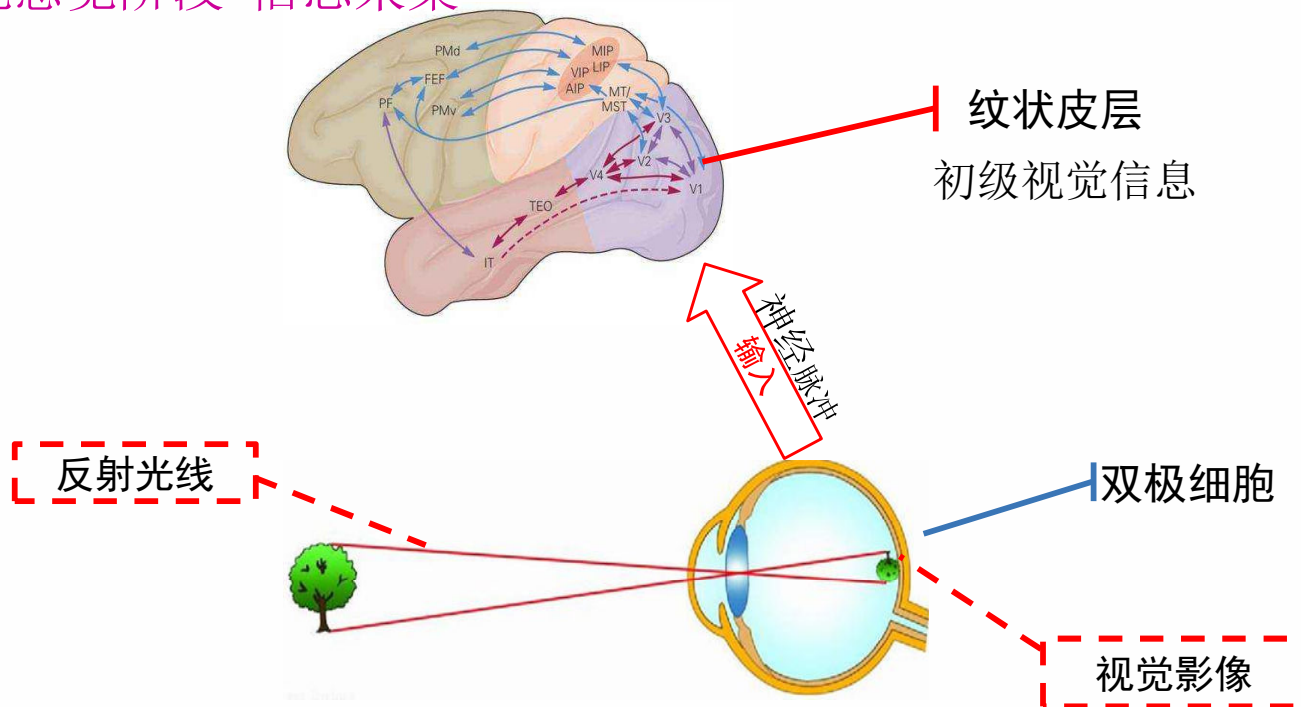
深度学习

- 手工地选取特征是一件非常费力、启发式（需要专业知识）的方法，能不能选取好很大程度上靠经验和运气，而且它的调节需要大量的时间

图像识别为什么用深度学习

人脑视觉机理

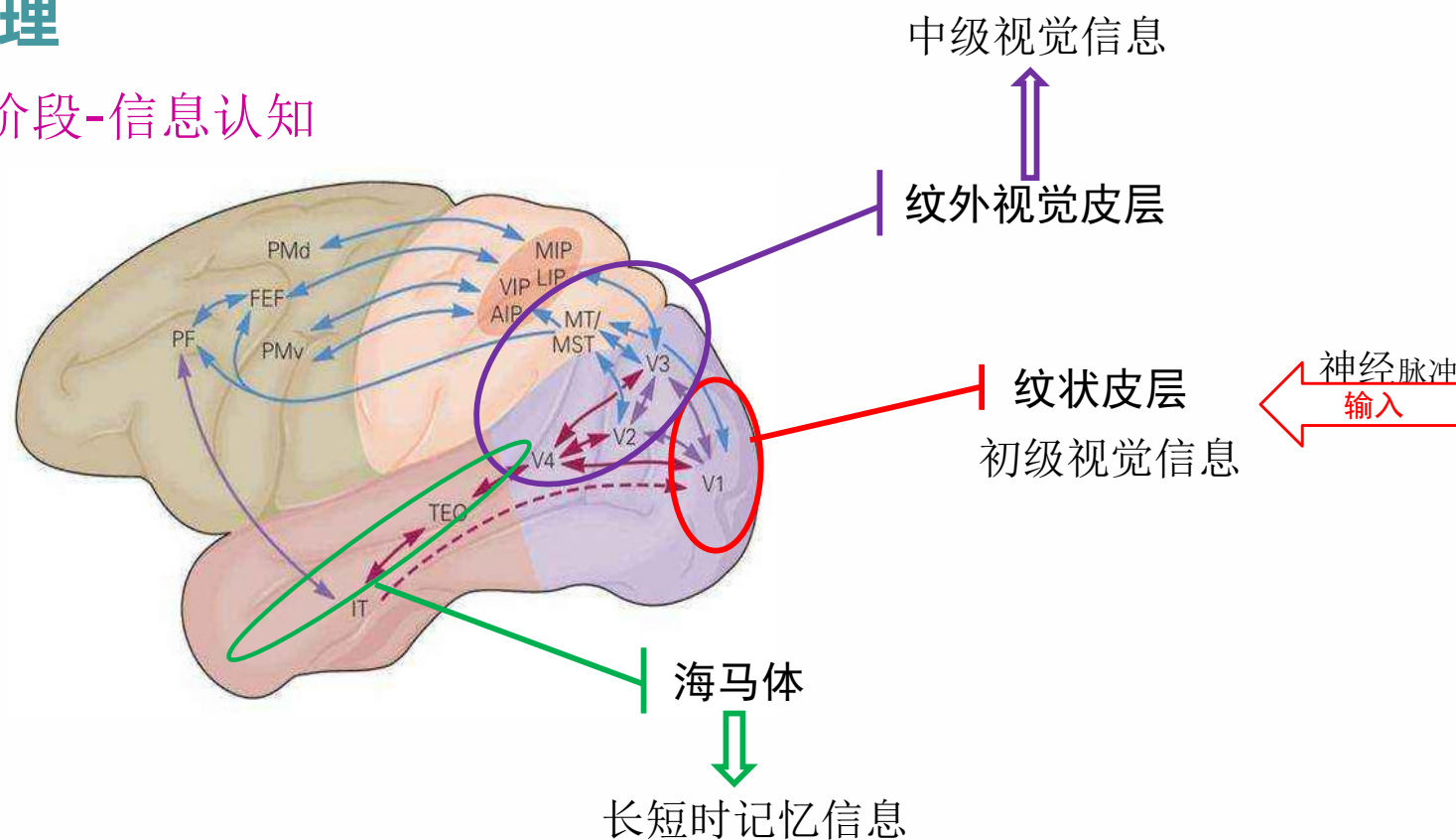
i. 视感觉阶段-信息采集



图像识别为什么用深度学习

人脑视觉机理

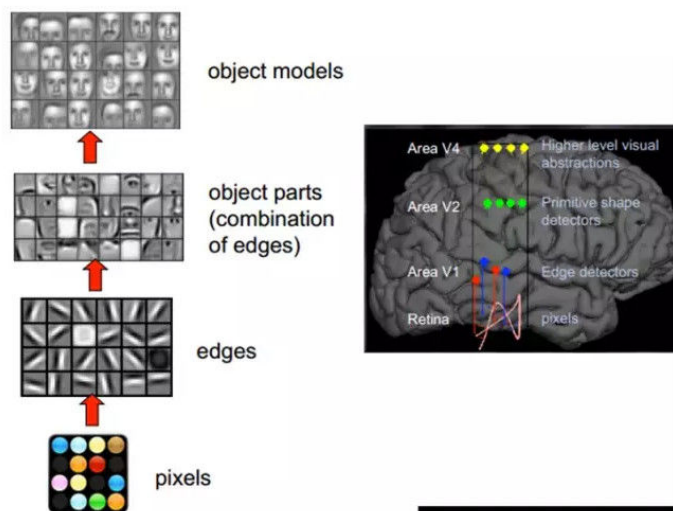
ii. 视知觉阶段-信息认知



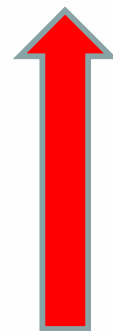
图像识别为什么用深度学习

人脑视觉机理：神经-中枢-大脑

- 从原始信号摄入（瞳孔摄入像素 Pixels）→初步处理（大脑皮层某些细胞发现边缘和方向）→抽象（大脑判定，眼前的物体的形状，是圆形的）→进一步抽象（大脑进一步判定该物体是人脸）
- 从原始信号，做低级抽象，逐渐向高级抽象迭代



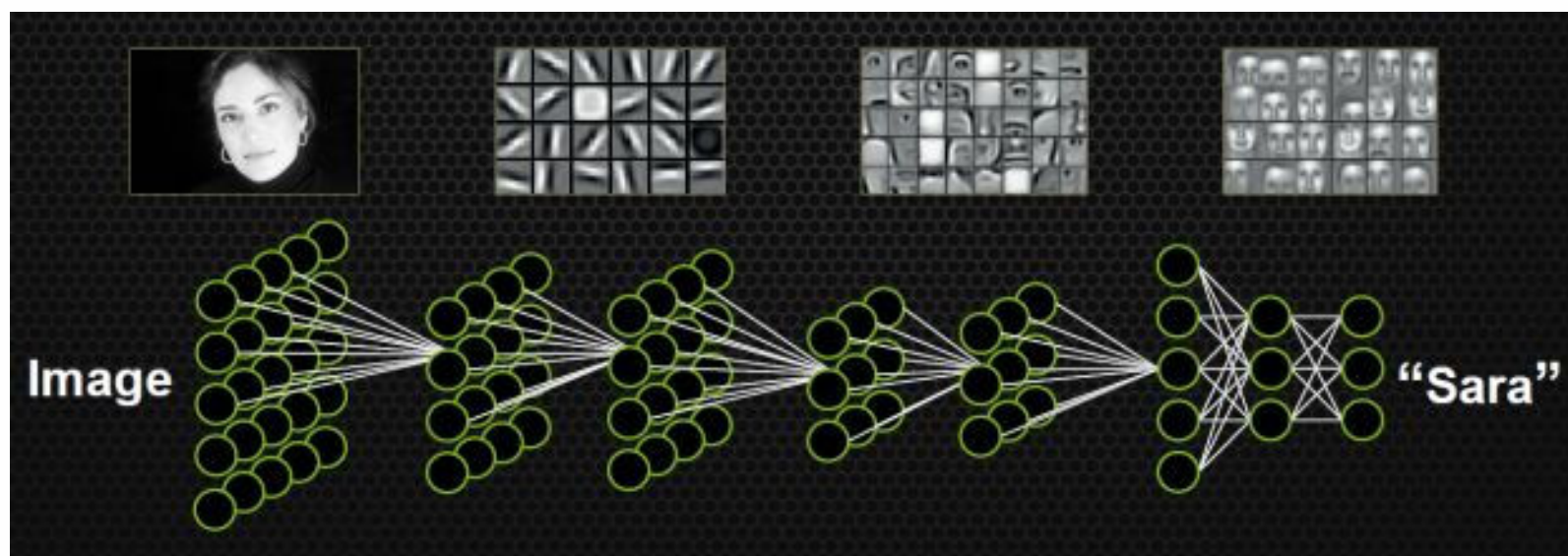
特征空间（概念）



测量空间（表象）

图像识别为什么用深度学习

深度学习模仿人类的视觉系统



深度学习与图像识别

深度学习发展历程

为什么使用深度学习

如何使用深度学习

如何使用深度学习解决图像识别

◆ 使用机器学习（深度学习）的目的：寻找一个合适的函数

- 数字识别

$$f(\text{) = \text{"3"}$$

- 猫狗识别

$$f(\text{) = \text{"cat"}$$

- 人脸识别

$$f(\text{) = \text{"隔壁老王"}$$

如何使用深度学习解决图像识别

◆ 图像识别

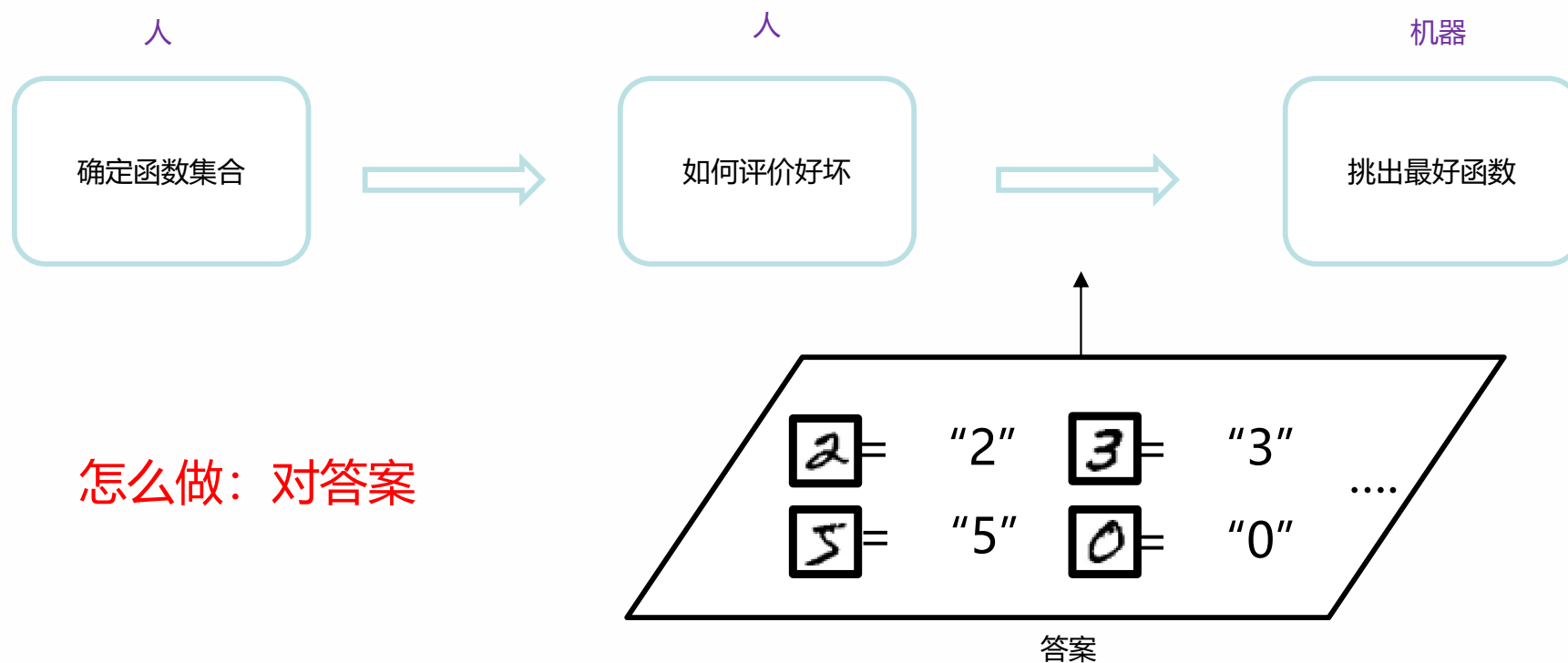
学习任务: 寻找手写识别函数, 候选函数集合为 $f, g, h \dots$

$$f\left(\boxed{\text{2}}\right) = \text{"烫烫烫烫"} \qquad g\left(\boxed{\text{2}}\right) = \text{"8"}$$

$$h\left(\boxed{\text{2}}\right) = \text{"2"}$$

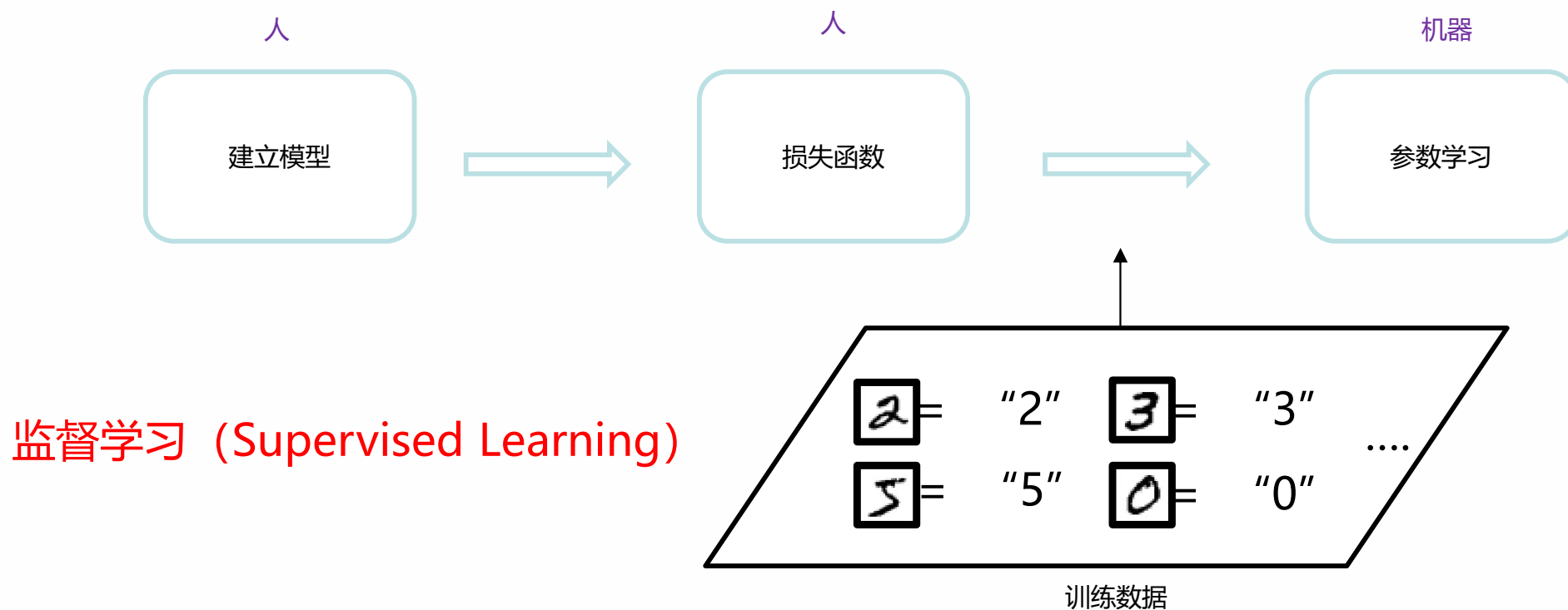
如何使用深度学习解决图像识别

◆ 学习过程



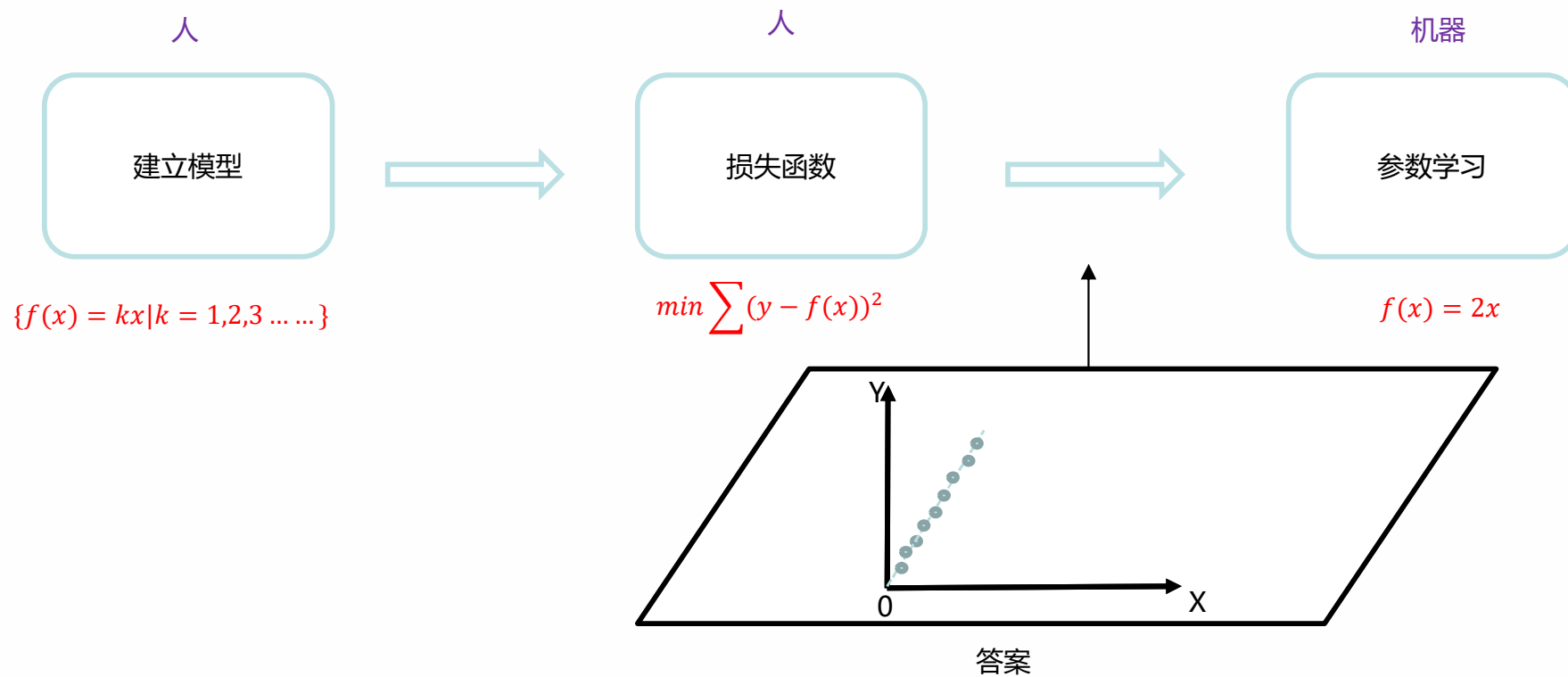
如何使用深度学习解决图像识别

◆ 学习过程



如何使用深度学习解决图像识别

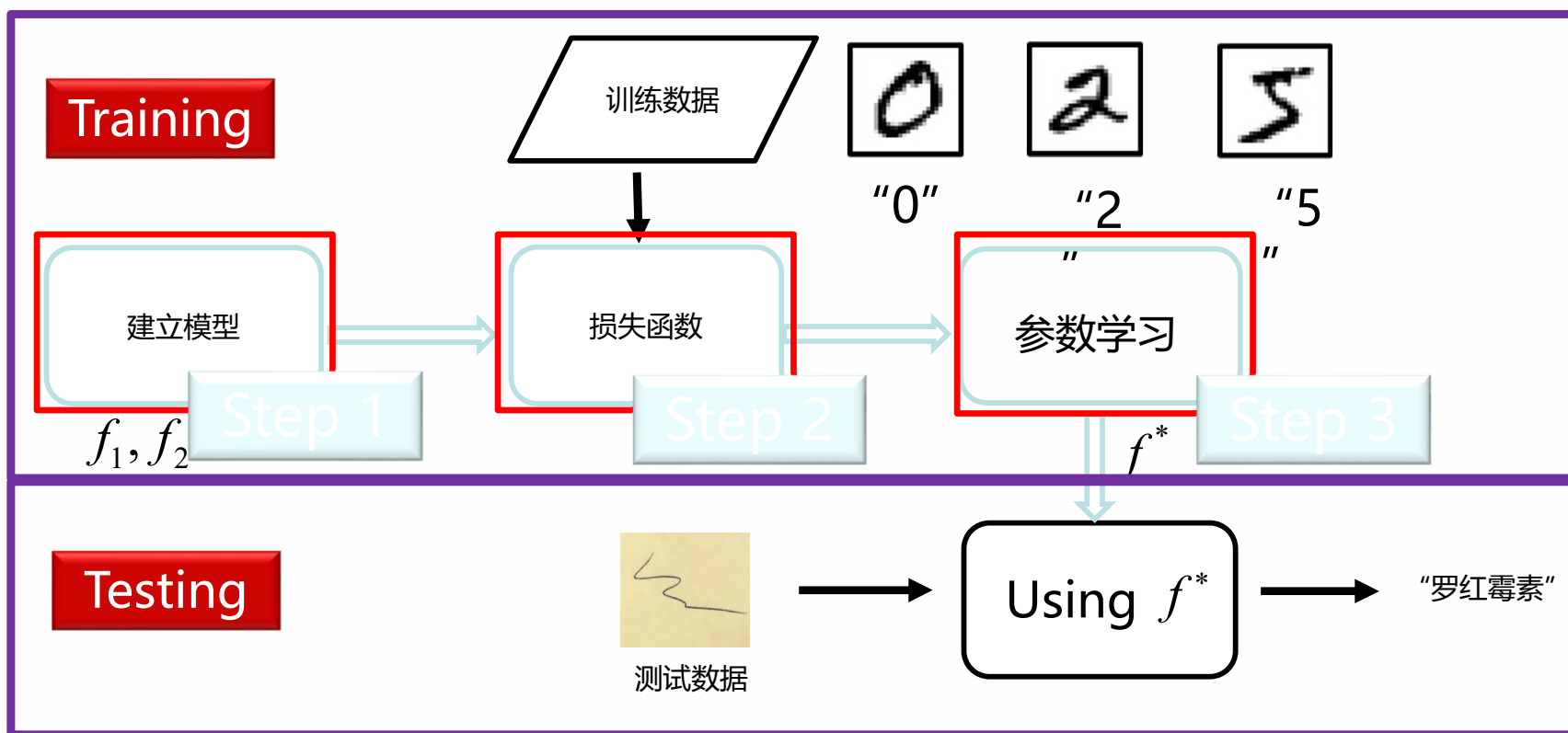
◆ 图像识别



如何使用深度学习解决图像识别

监督学习任务: 手写识别

◆ 图像识别



使用步骤

★ 建立模型

- 选择什么样的网络结构
- 选择多少层数，每层选择多少神经元

★ 损失函数

- 选择常用损失函数，平方误差，交叉熵....

★ 参数学习

- 梯度下降
- 反向传播算法

使用步骤

★ 建立模型

- 选择什么样的网络结构
- 选择多少层数，每层选择多少神经元

★ 损失函数

- 选择常用损失函数，平方误差，交叉熵....

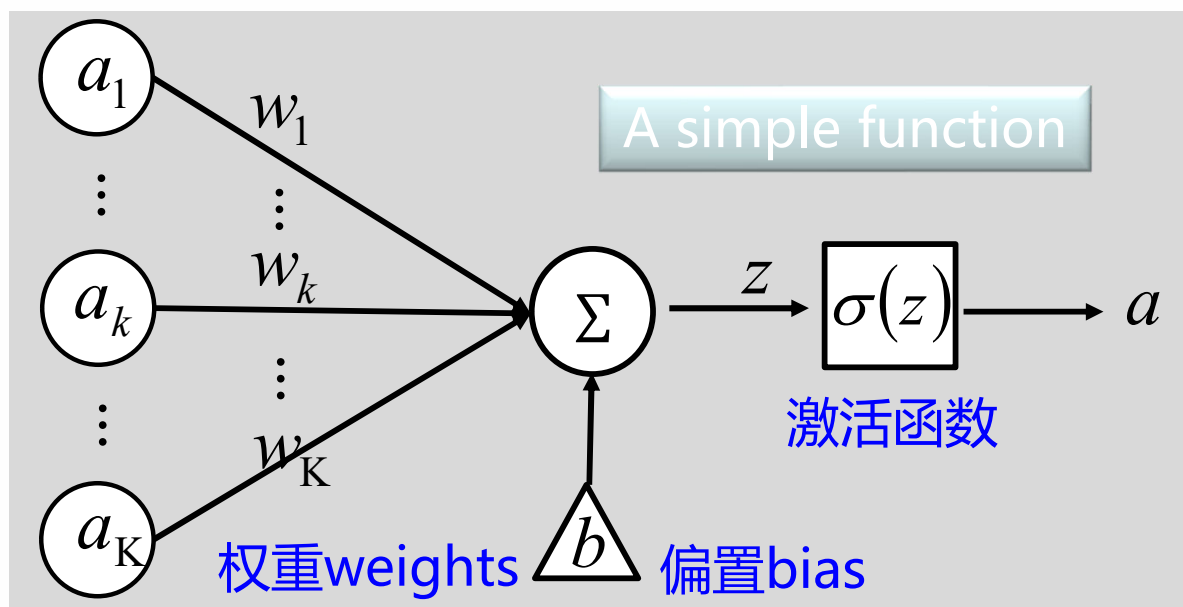
★ 参数学习

- 梯度下降
- 反向传播算法

建立模型

★ 神经元

$$z = a_1 w_1 + \cdots + a_k w_k + \cdots + a_K w_K + b$$



理解：简单线性函数

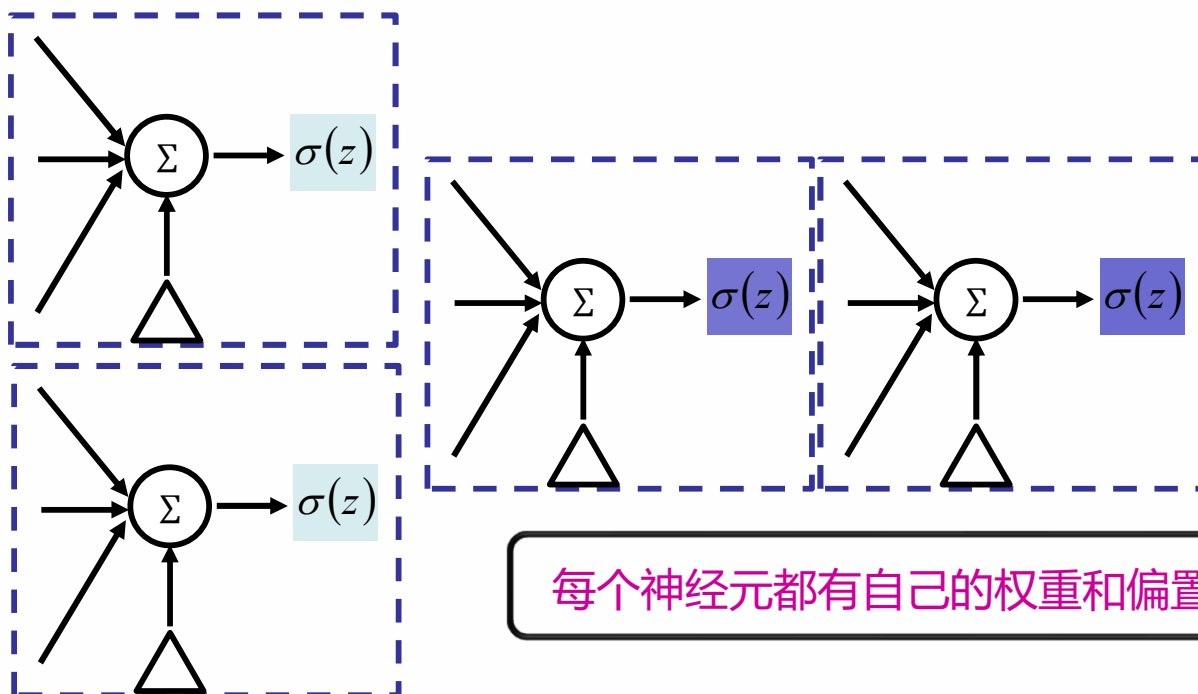
$$f(x) = kx + b$$

(k是斜率, b是截距)

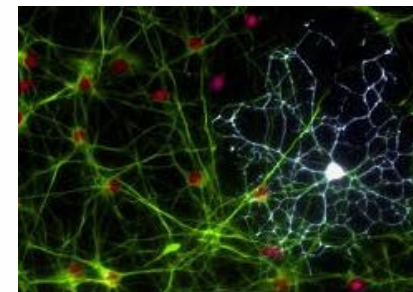
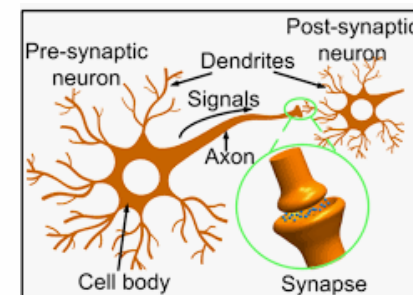
建立模型

★ 网络结构

◆ 神经元不同的连接方式构成不同的网络结构



每个神经元都有自己的权重和偏置参数

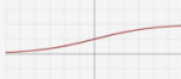
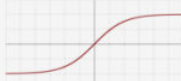
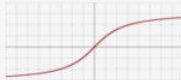
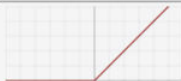
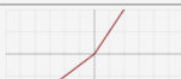


建立模型

★ 激活函数

◆ 为什么引入激活函数

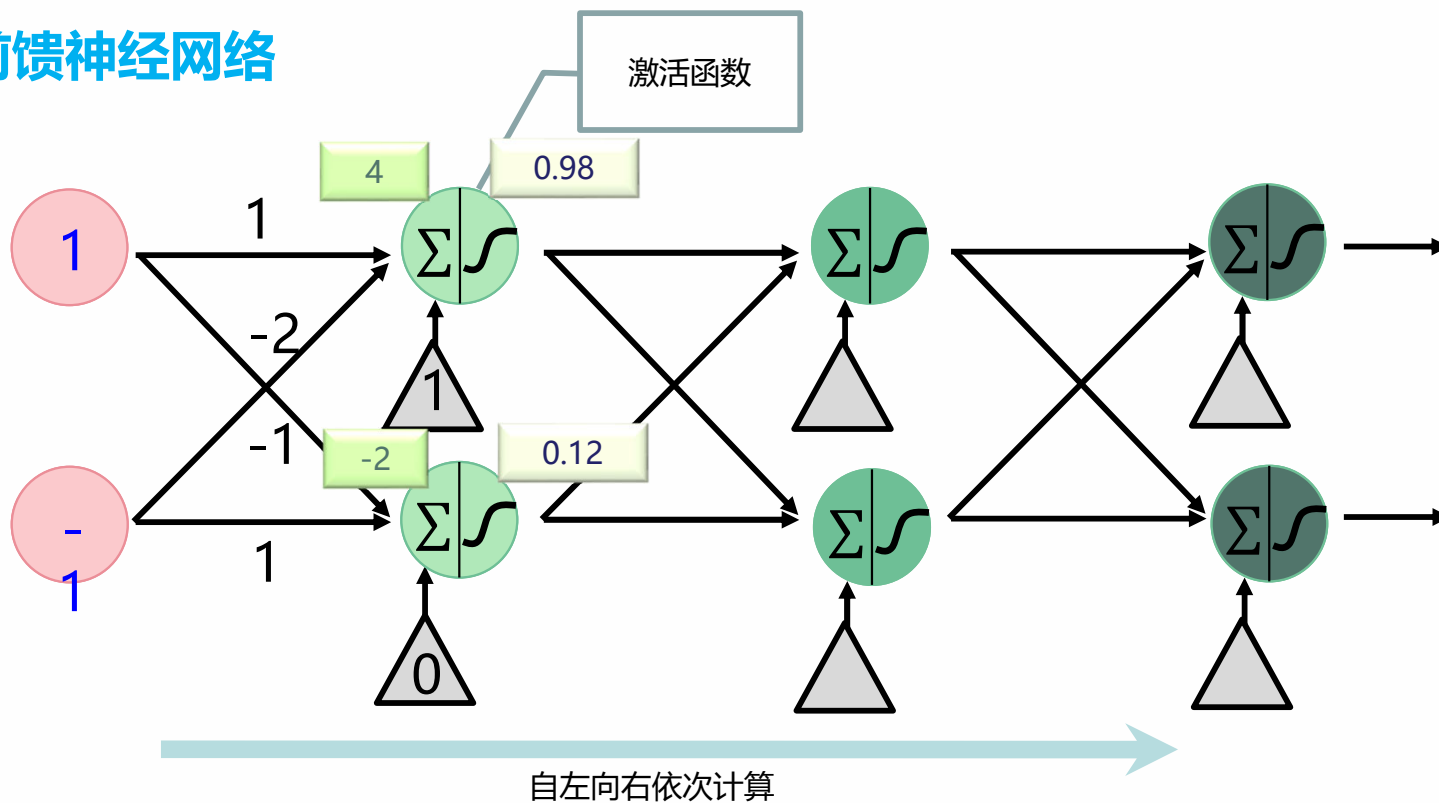
- 为了增强网络的表达能力，我们需要激活函数来将线性函数->非线性函数
- 非线性的激活函数需要有连续性。因为连续非线性激活函数可以可导的，所以可以用最优化的方法来求解

Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$
TanH		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$
Rectified Linear Unit (ReLU) ^[7]		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parameteric Rectified Linear Unit (PReLU) ^[8]		$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$

常用激活函数示例

建立模型

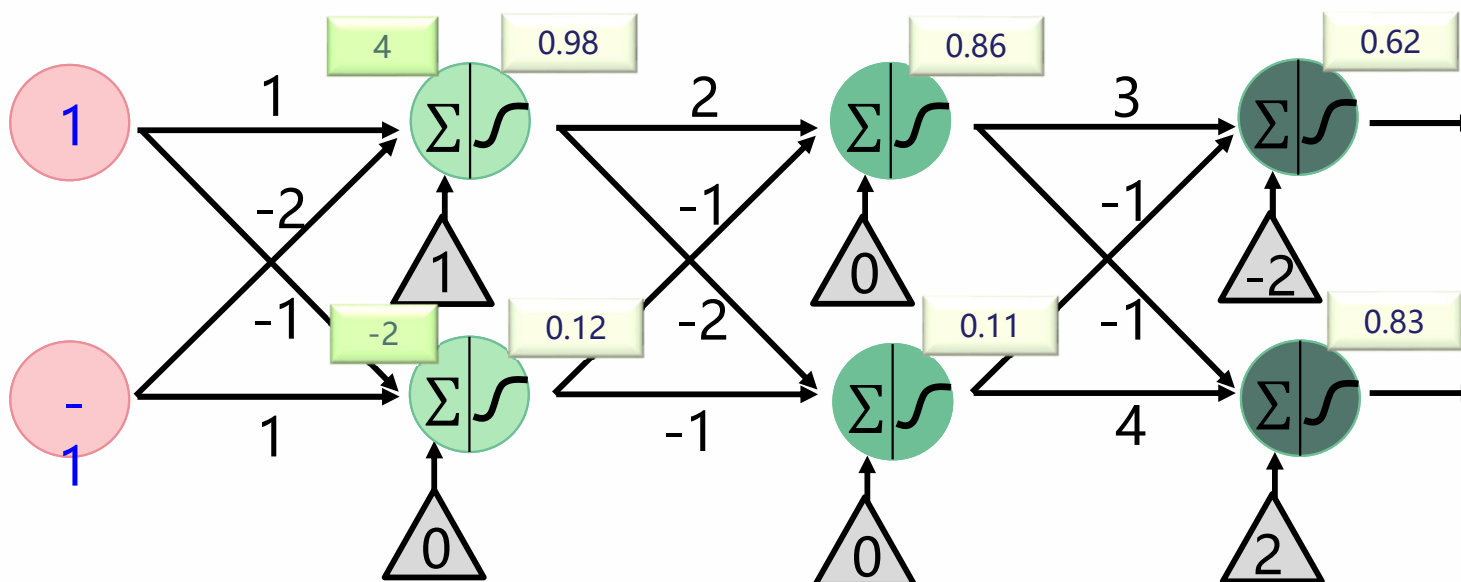
★ 前馈神经网络



建立模型



前馈神经网络

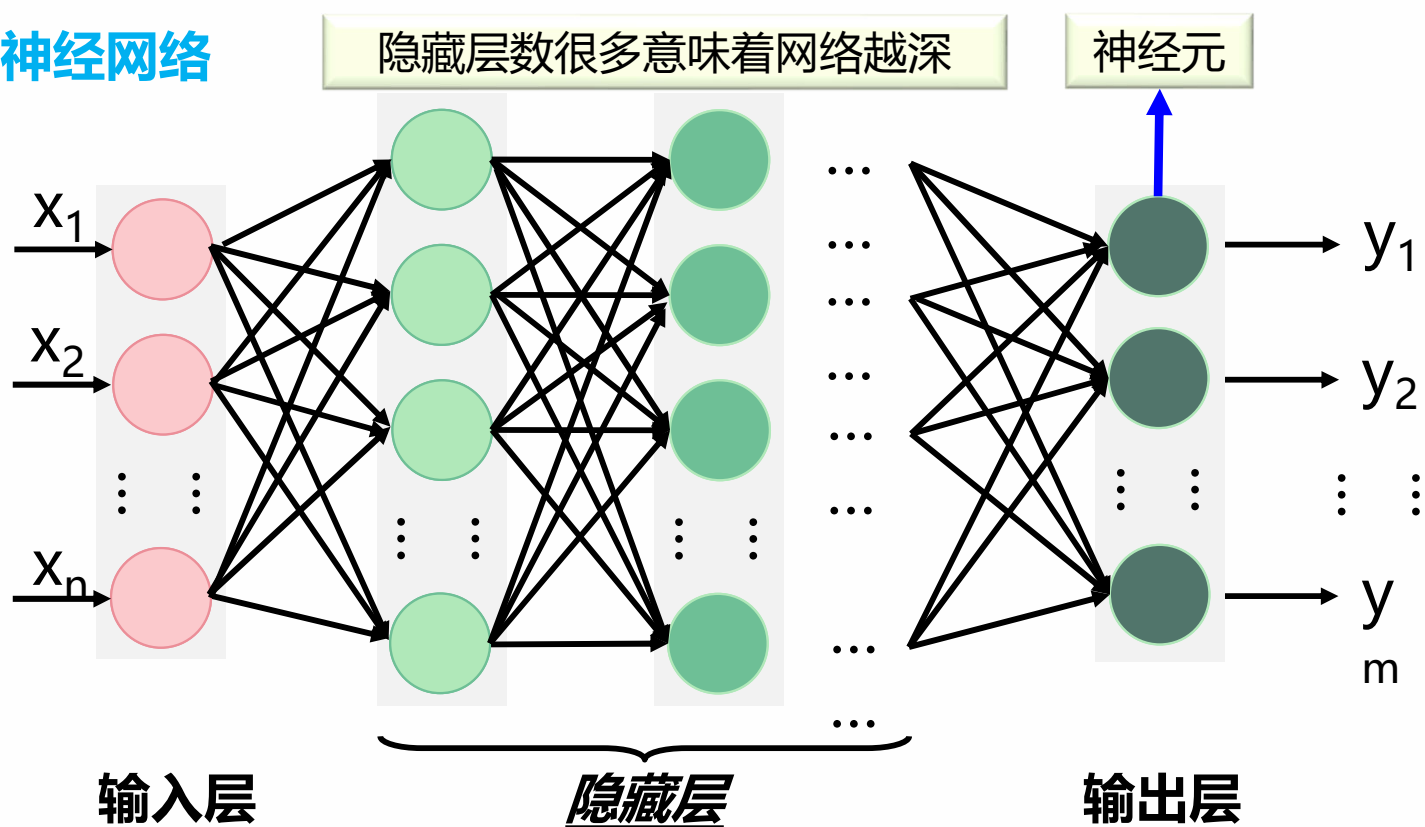


函数: $f\left(\begin{bmatrix} 1 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} 0.62 \\ 0.83 \end{bmatrix}$

建立模型

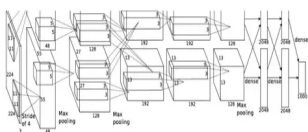
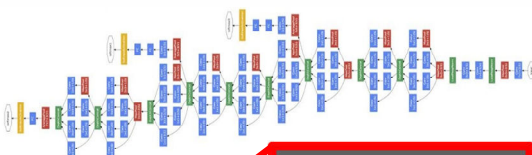
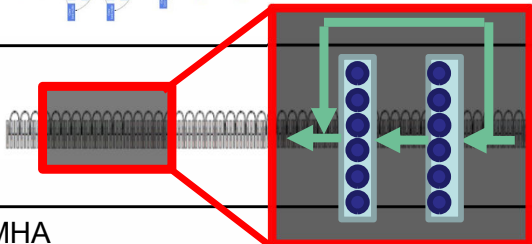


前馈神经网络



建立模型

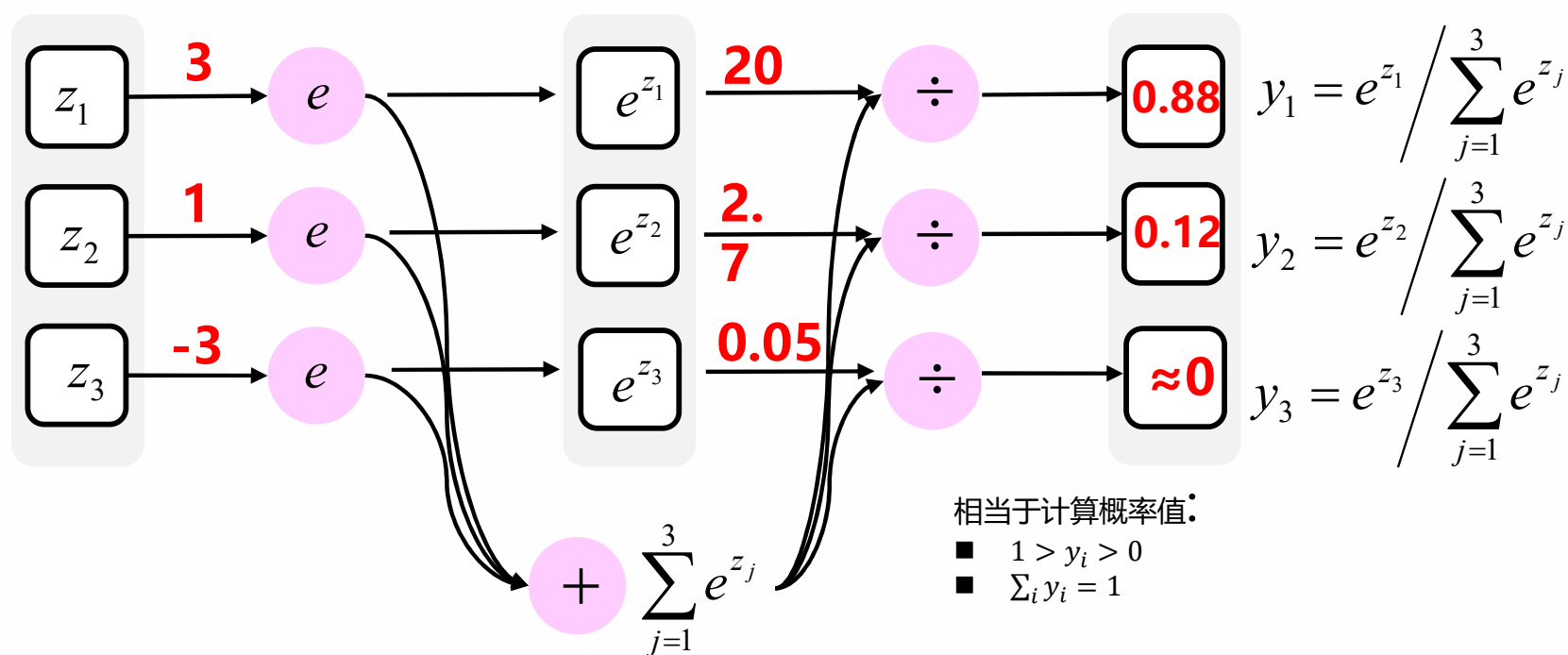
★ 例子

模型	层数	结构	错误率
AlexNet (2012)	8层		16.4%
VGG (2014)	19层		7.3%
GoogleNet (2014)	22层		6.7%
Residual Net (2015)	152层		3.57%
ViT(Transformer)	Larger	MHA	Better

建立模型

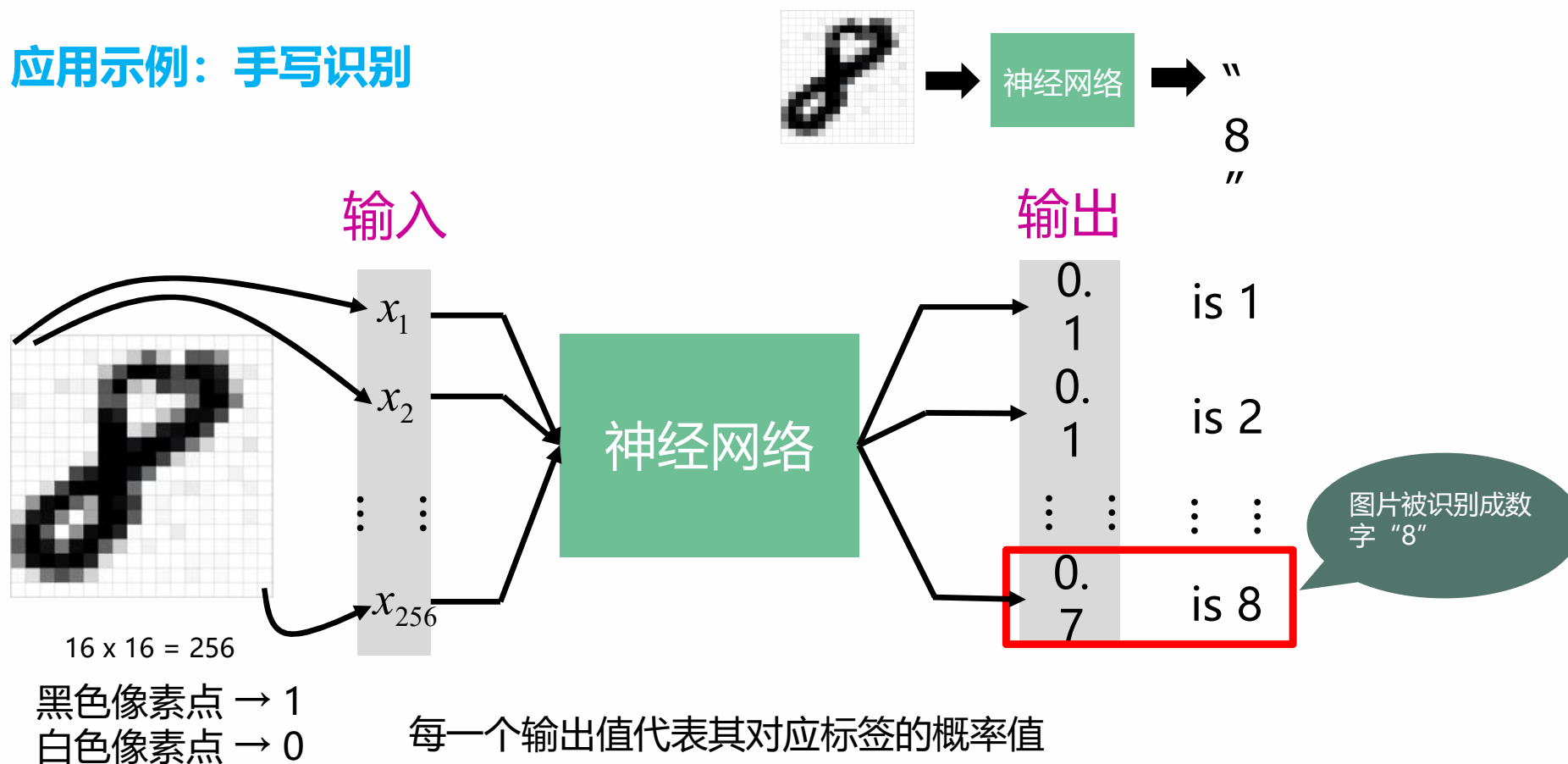
★ 输出层

- 常用softmax函数作为输出层激活函数：容易理解、便于计算



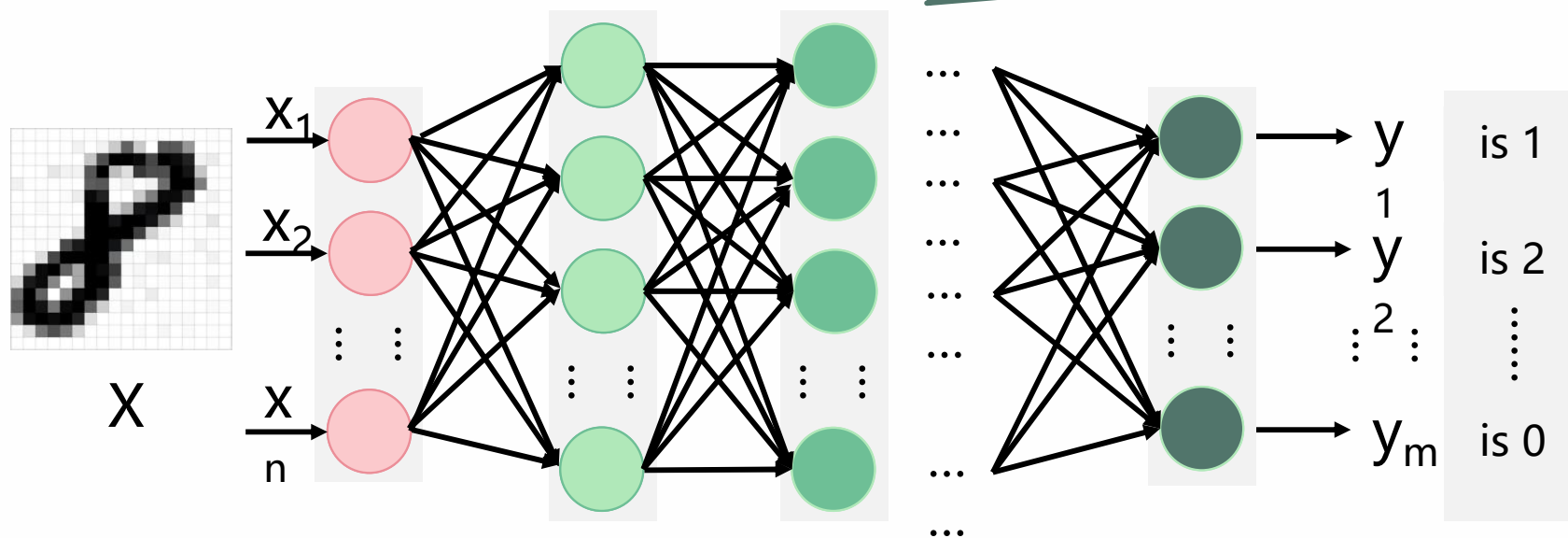
建立模型

★ 应用示例：手写识别



建立模型

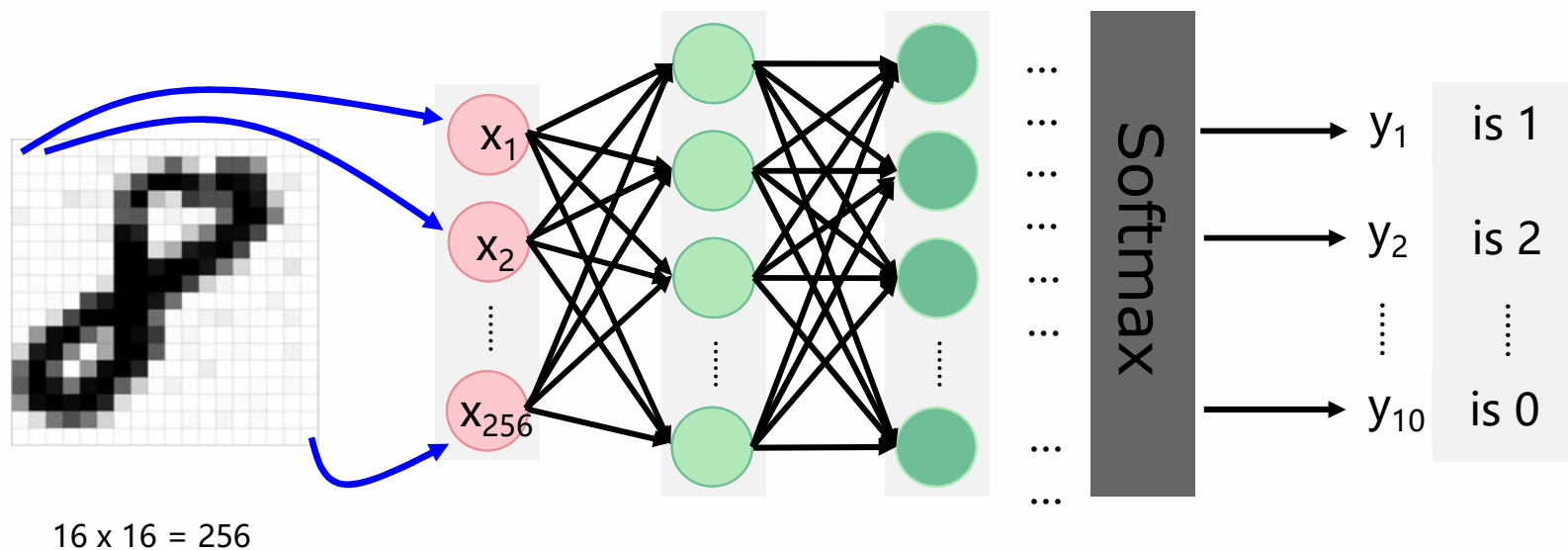
★ 应用示例：手写体识别



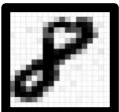
问题：应该设置多少层，多少结点？
是否需要选择其他网络结构如CNN/RNN？

设置合适的网络结构：层数和结点个数、激活函数

损失函数



对于数字识别任务，设计用于分类的损失函数，使得学习目标变为：

输入:  $\rightarrow y_1$ 值最大; 输入:  $\rightarrow y_8$ 值最大

使用步骤

★ 建立模型

- 选择什么样的网络结构
- 选择多少层数，每层选择多少神经元

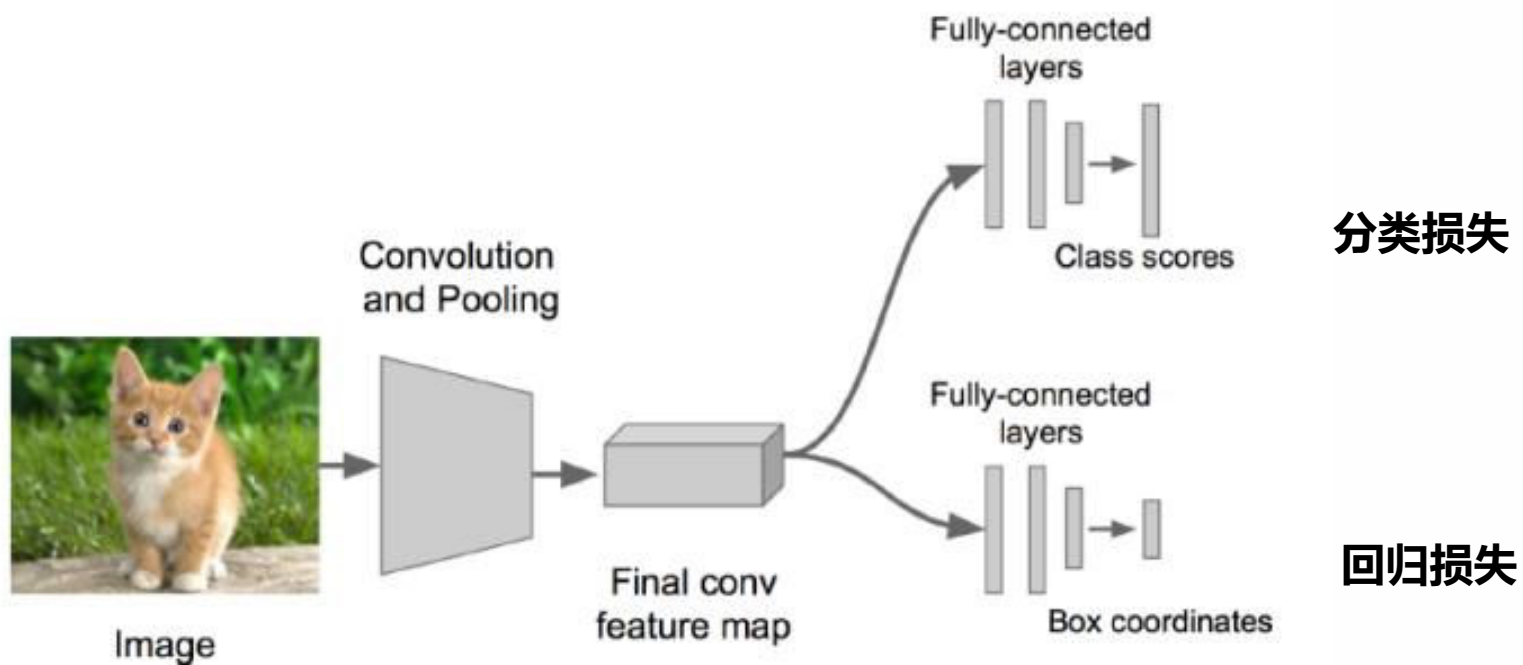
★ 损失函数

- 选择常用损失函数，平方误差，交叉熵....

★ 参数学习

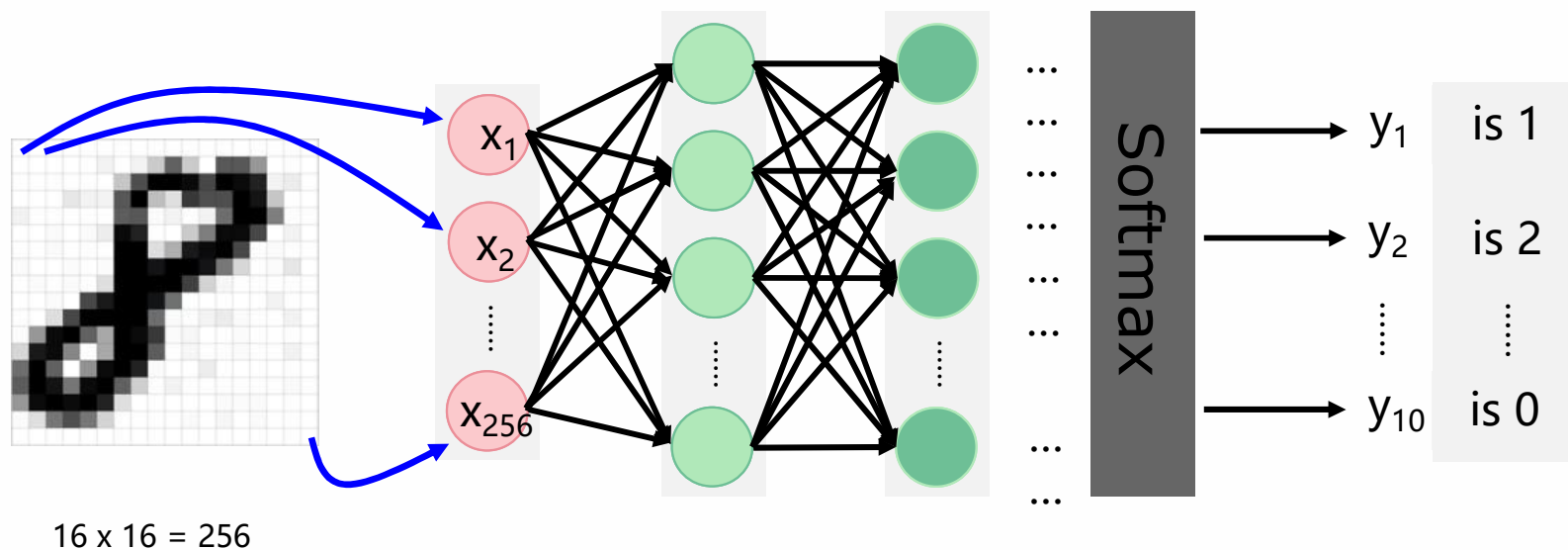
- 梯度下降
- 反向传播算法

损失函数

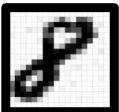


损失函数的设计依赖于具体的任务

损失函数

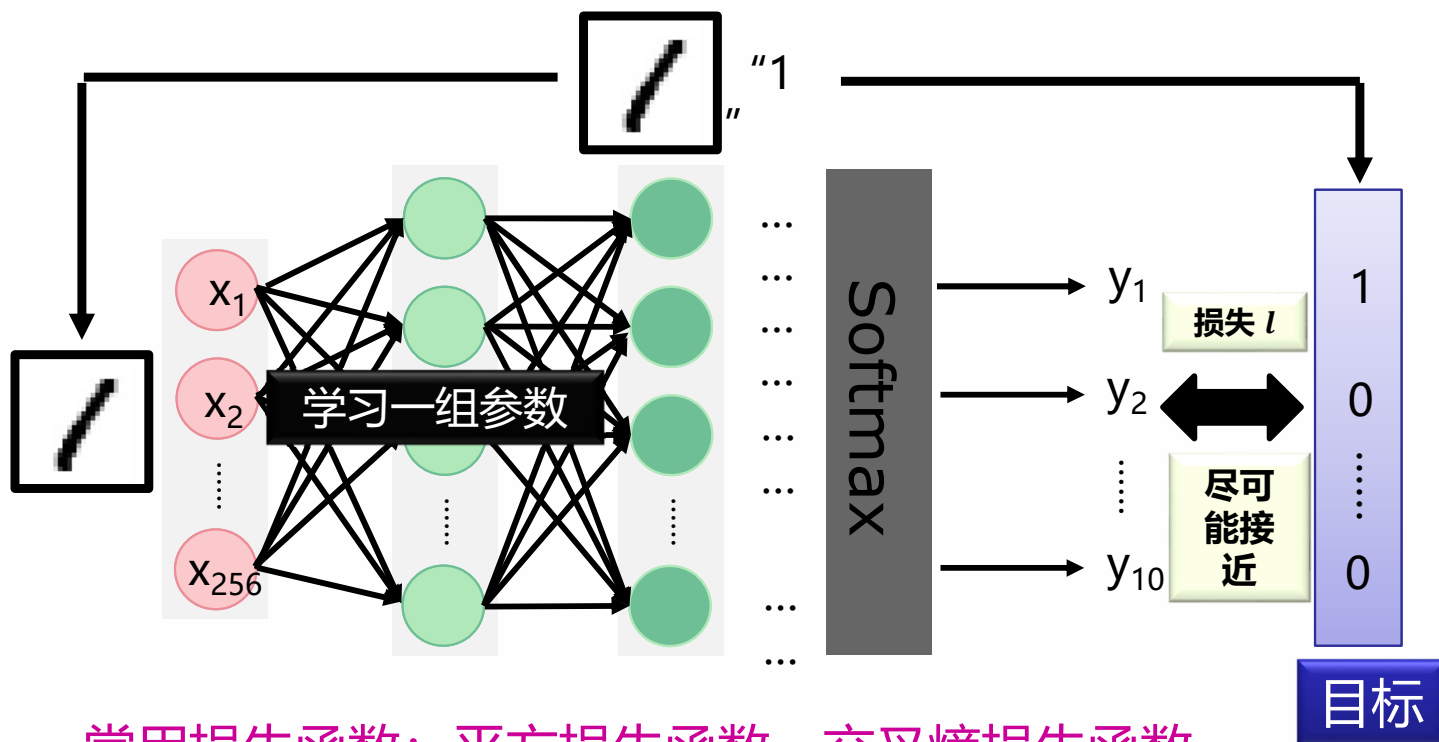


对于数字识别任务，设计用于分类的损失函数，使得学习目标变为：

输入:  $\rightarrow y_1$ 值最大; 输入:  $\rightarrow y_8$ 值最大

损失函数

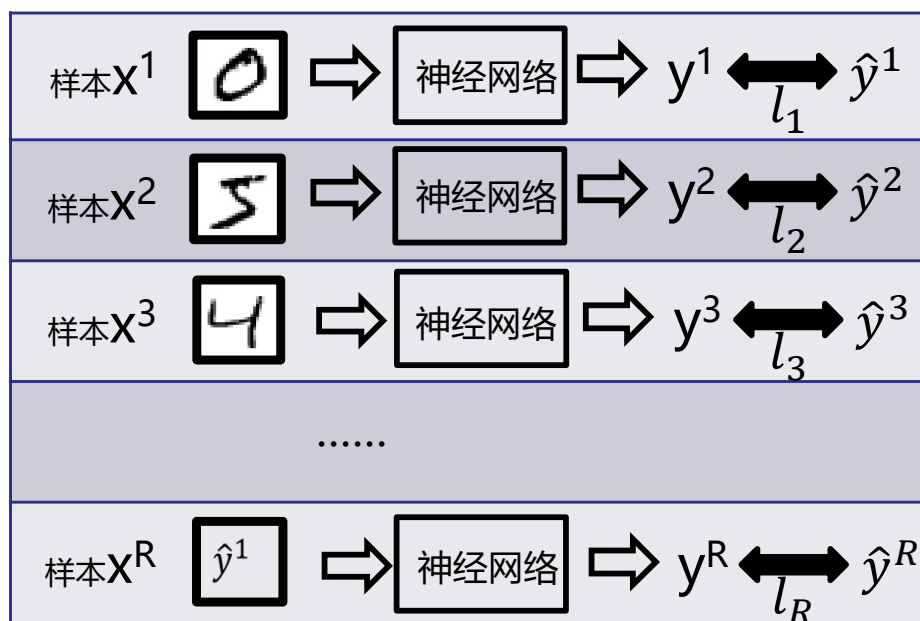
好的参数使得所有训练数据的损失越小越好



常用损失函数：平方损失函数、交叉熵损失函数

损失函数

对所有训练数据:



总损失:

$$L = \sum_{r=1}^R l_r$$

尽可能小

找到一个函数使得总损失 L 最小

即确定参数使得总损失 L 最小

使用步骤

★ 建立模型

- 选择什么样的网络结构
- 选择多少层数，每层选择多少神经元

★ 损失函数

- 选择常用损失函数，平方误差，交叉熵....

★ 参数学习

- 梯度下降
- 反向传播算法

参数学习

- 枚举所有可能的取值

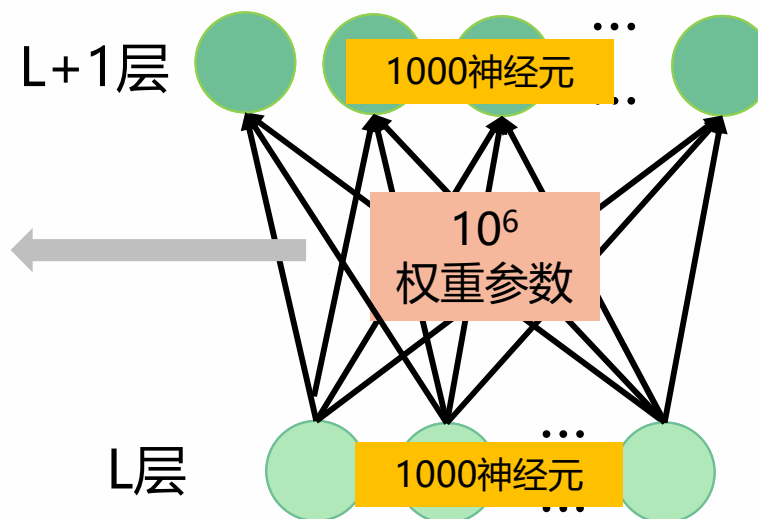


网络参数 $\theta =$

$$\{w_1, w_2, w_3, \dots, b_1, b_2, b_3, \dots\}$$

参数个数巨大

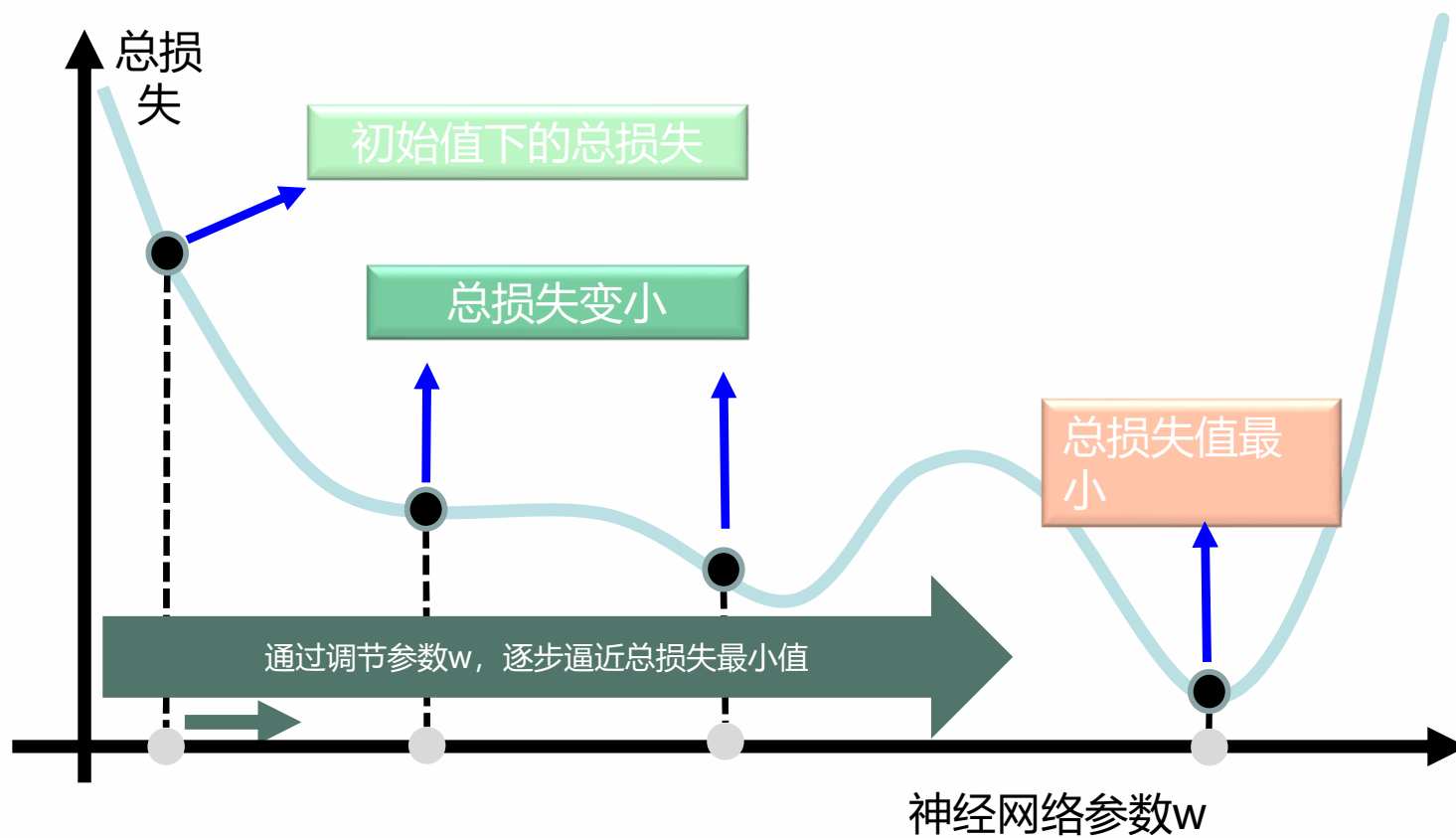
- 寻找模型参数 θ^* 使得总损失 L 最小



例如：语音识别模型有 8 层，每层 1000 神经元

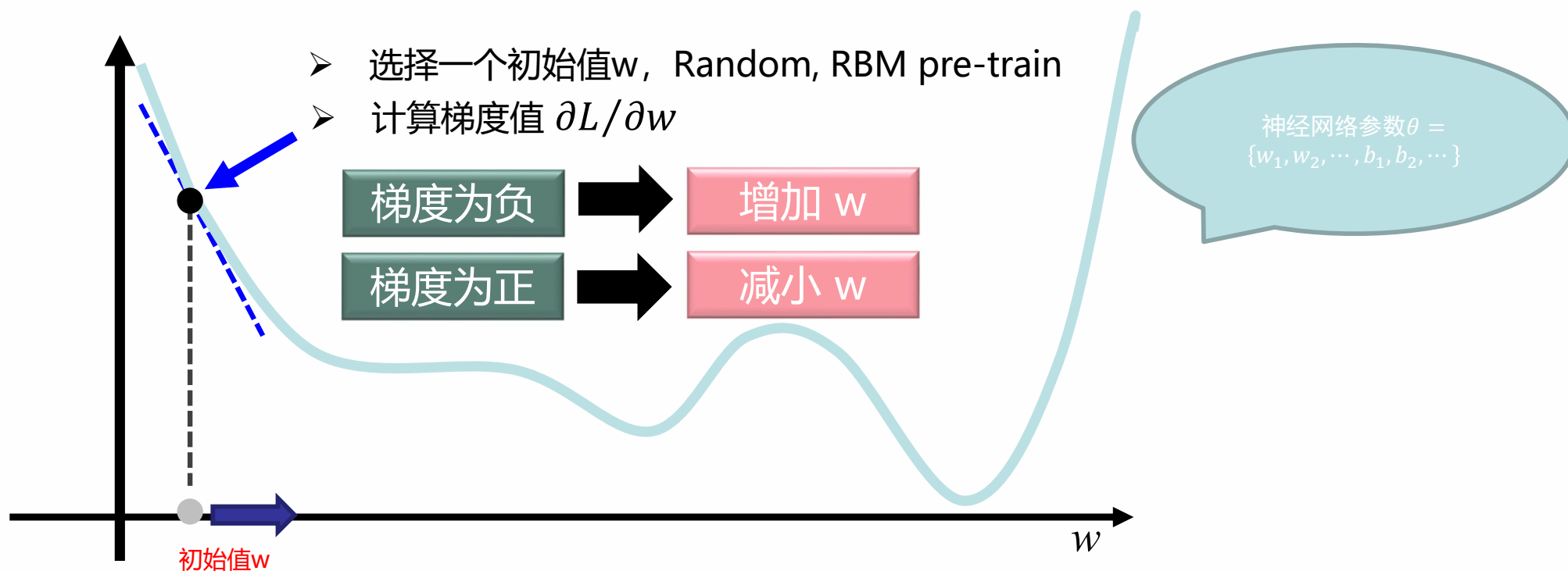
参数学习

★ 梯度下降法



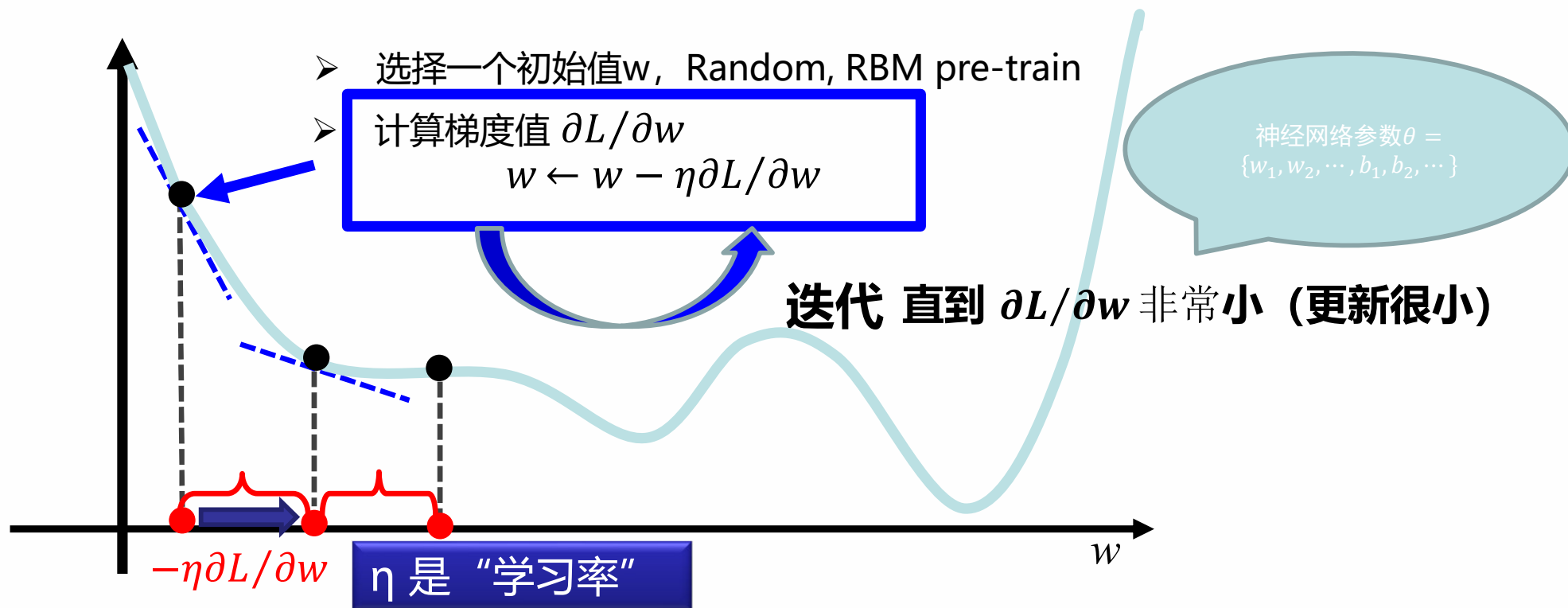
参数学习

★ 梯度下降法



参数学习

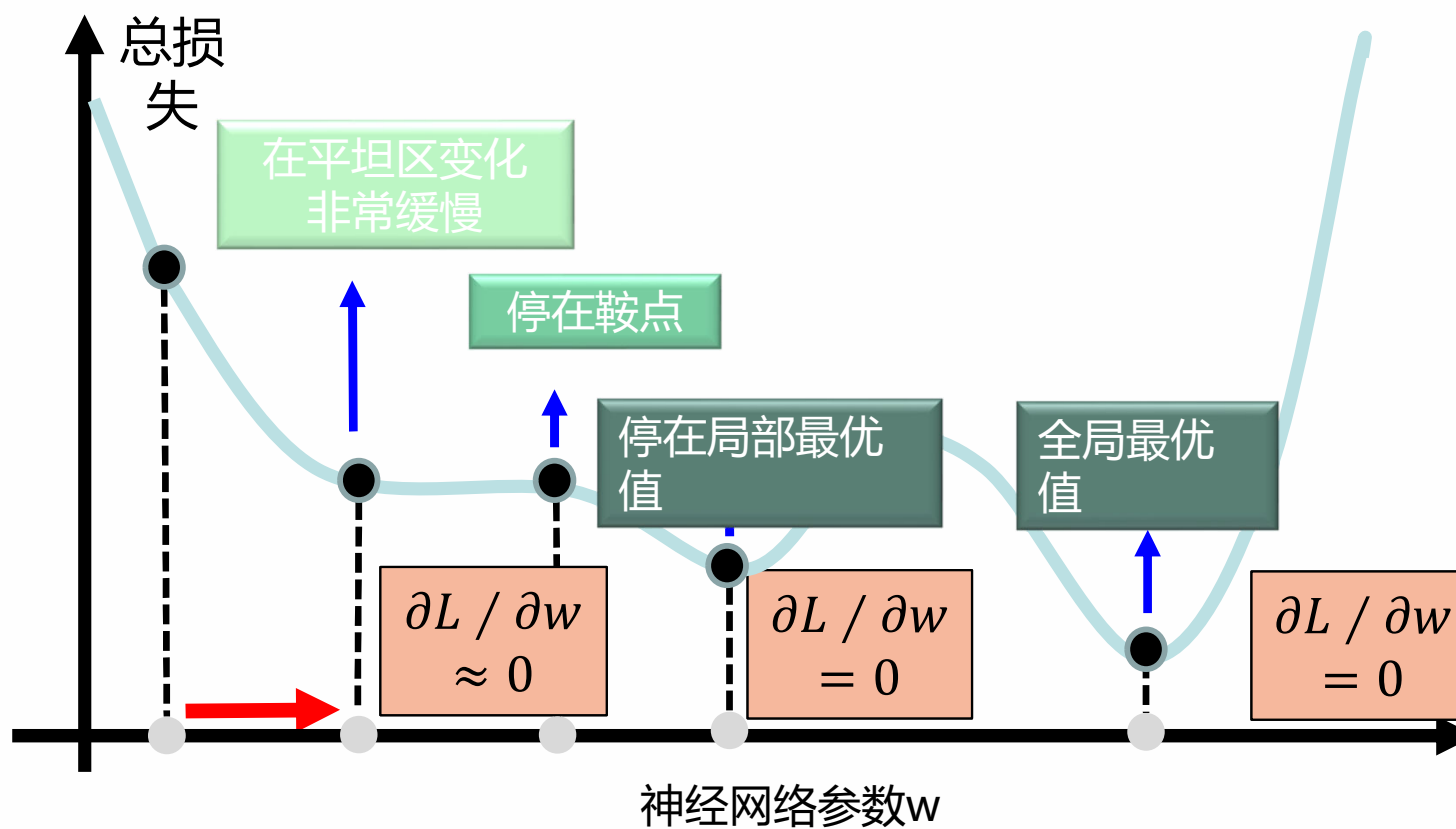
★ 梯度下降法



参数学习

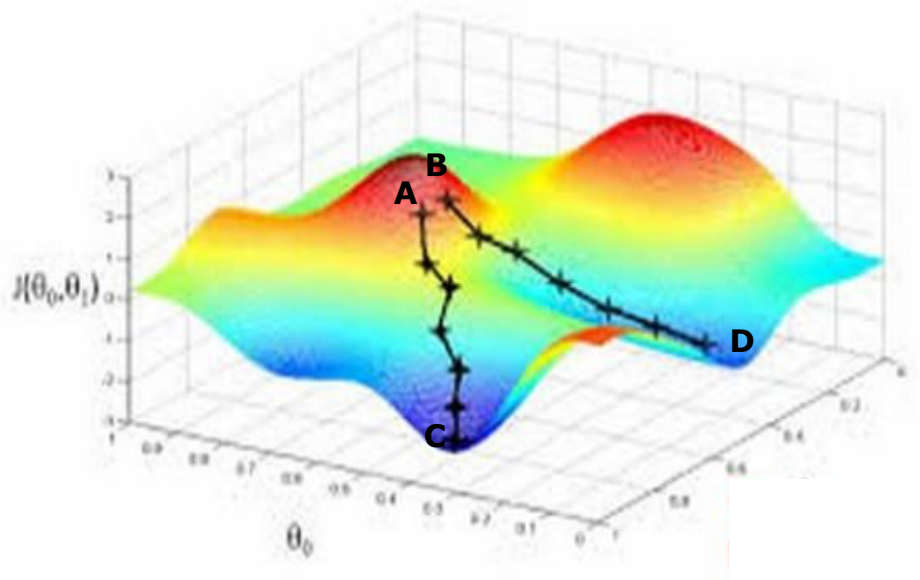


梯度下降法



参数学习

★ 初始值影响



例如：

A → C

B → D

选取不同的初始值，可能到达不同的局部最小值

使用步骤

★ 建立模型

- 选择什么样的网络结构
- 选择多少层数，每层选择多少神经元

★ 损失函数

- 选择常用损失函数，平方误差，交叉熵....

★ 参数学习

- 梯度下降
- 反向传播算法

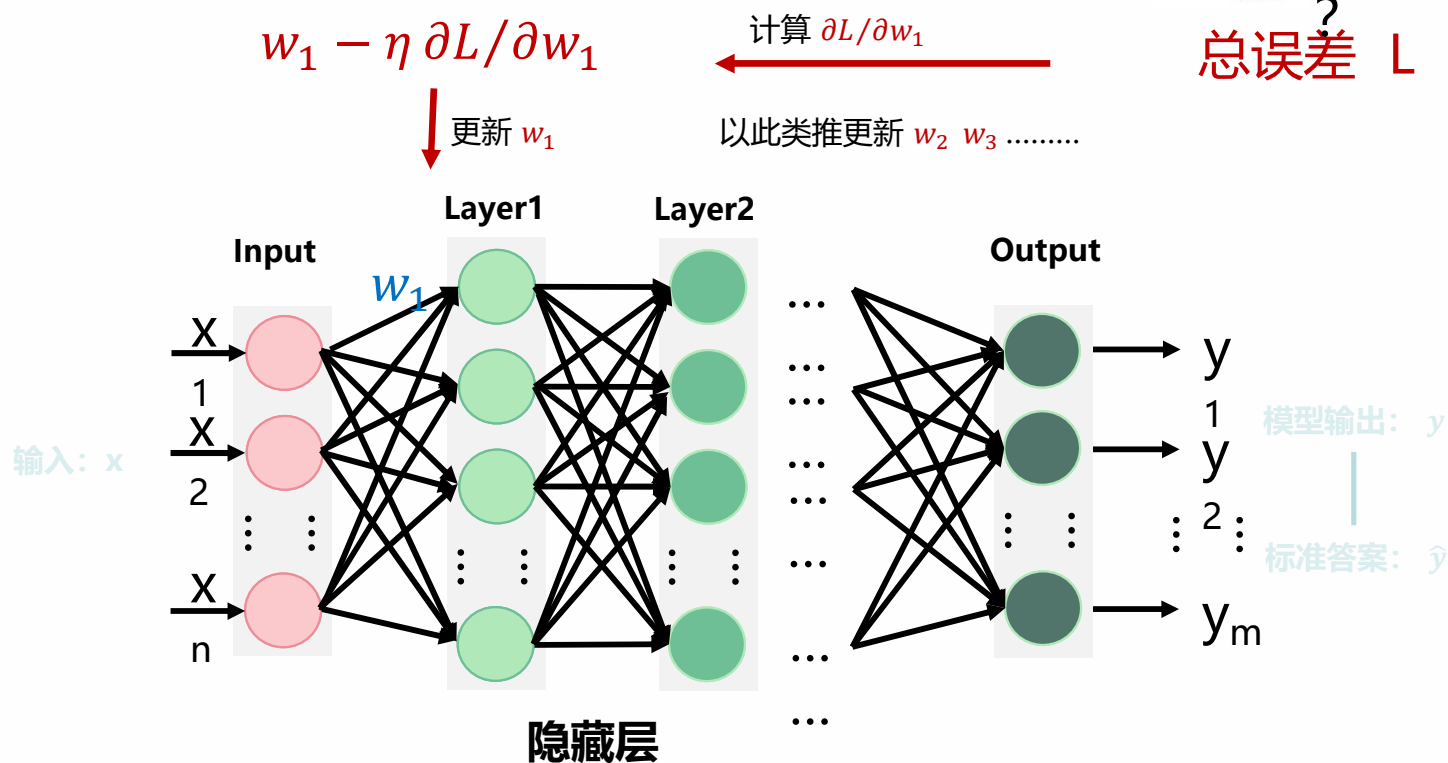
参数学习

★ 反向传播算法



只剩一个问题：
怎么求 梯度 $\partial L / \partial w$

总误差 L

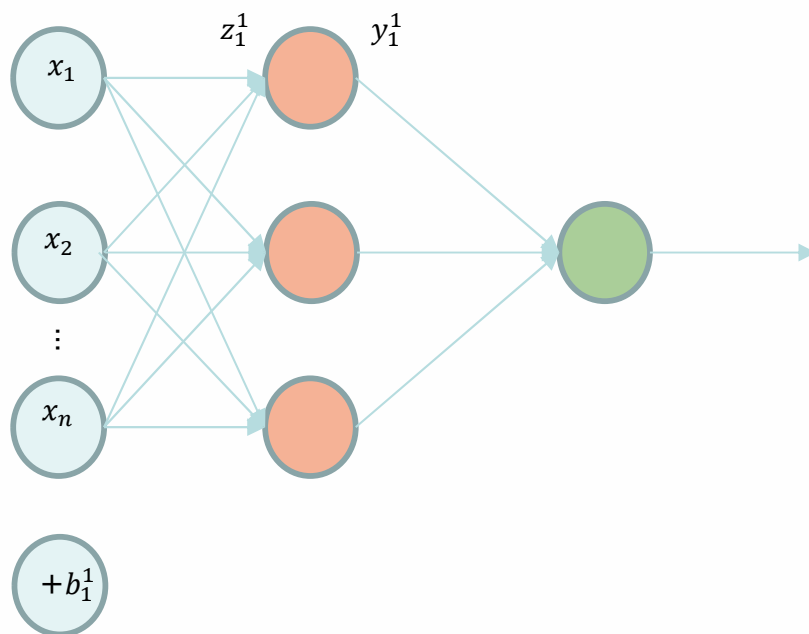


参数学习



信号正向传播

- 假定第 l 层神经元状态为 z^l ，经激活函数后的输出值为 y^l

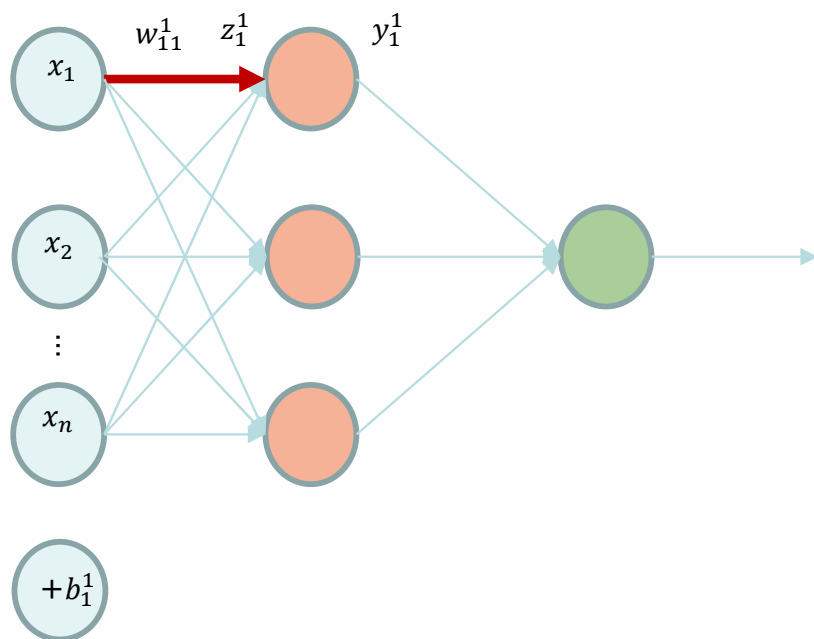


参数学习



信号正向传播

➤ 假定第 l 层神经元状态为 z^l ，经激活函数后的输出值为 y^l



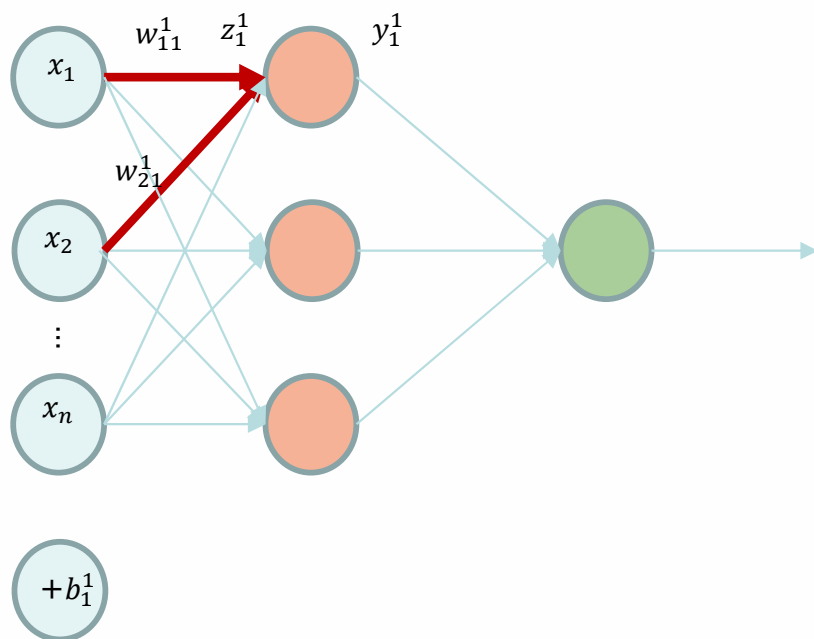
- $z_1^1 = w_{11}^1 x_1$

参数学习



信号正向传播

➤ 假定第 l 层神经元状态为 z^l ，经激活函数后的输出值为 y^l

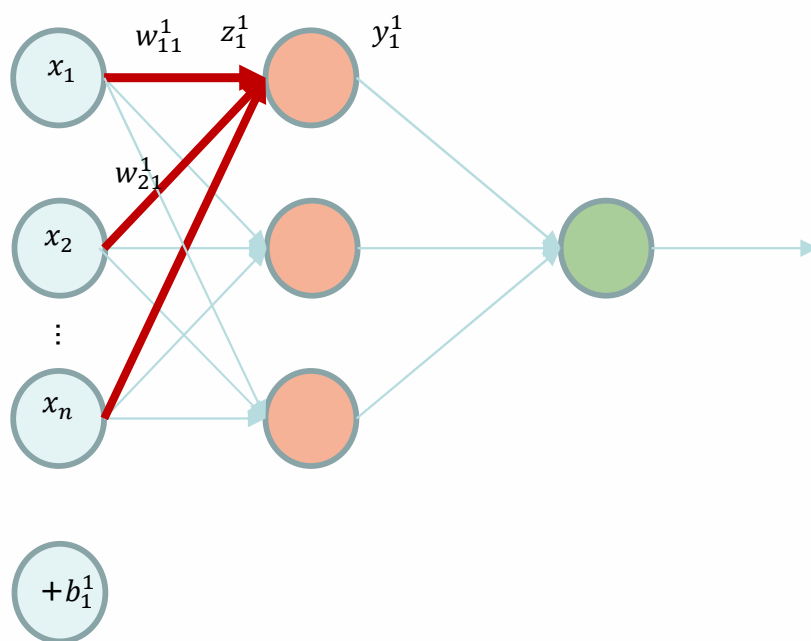


$$\bullet \quad z_1^1 = w_{11}^1 x_1 + w_{21}^1 x_2$$

参数学习

★ 信号正向传播

➤ 假定第 l 层神经元状态为 z^l ，经激活函数后的输出值为 y^l



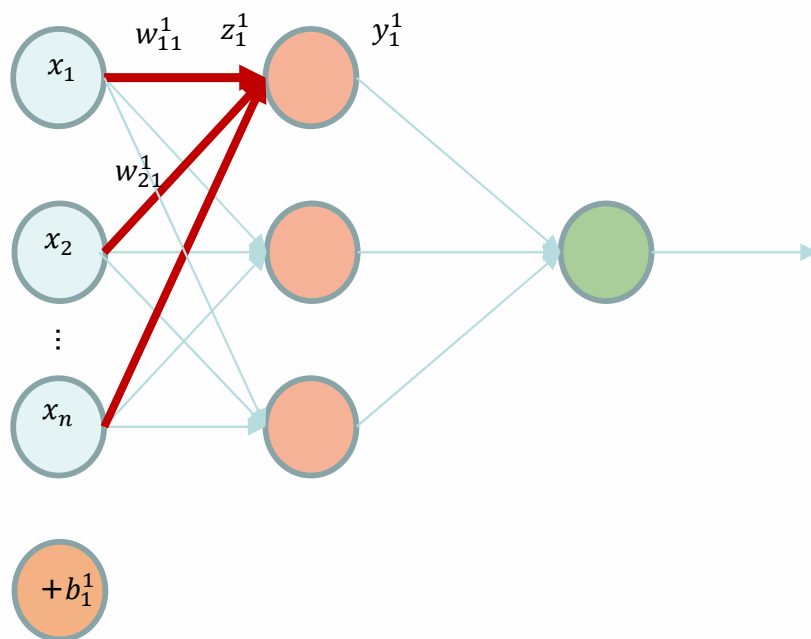
$$\bullet \quad z_1^1 = w_{11}^1 x_1 + w_{21}^1 x_2 + \dots + w_{n1}^1 x_n$$

参数学习



信号正向传播

➤ 假定第 l 层神经元状态为 z^l ，经激活函数后的输出值为 y^l



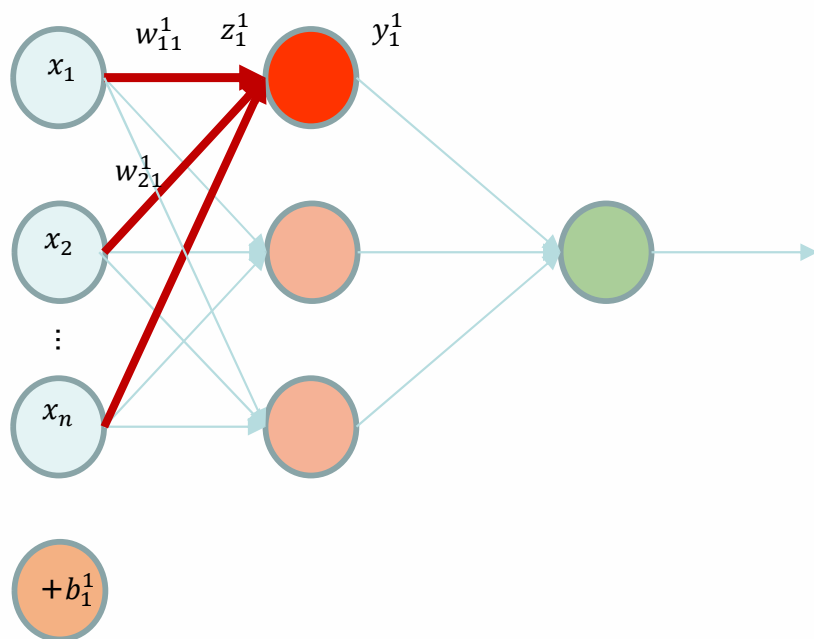
$$\bullet \quad z_1^1 = w_{11}^1 x_1 + w_{21}^1 x_2 + \dots + w_{n1}^1 x_n + b_1^1$$

参数学习



信号正向传播

➤ 假定第 l 层神经元状态为 z^l ，经激活函数后的输出值为 y^l



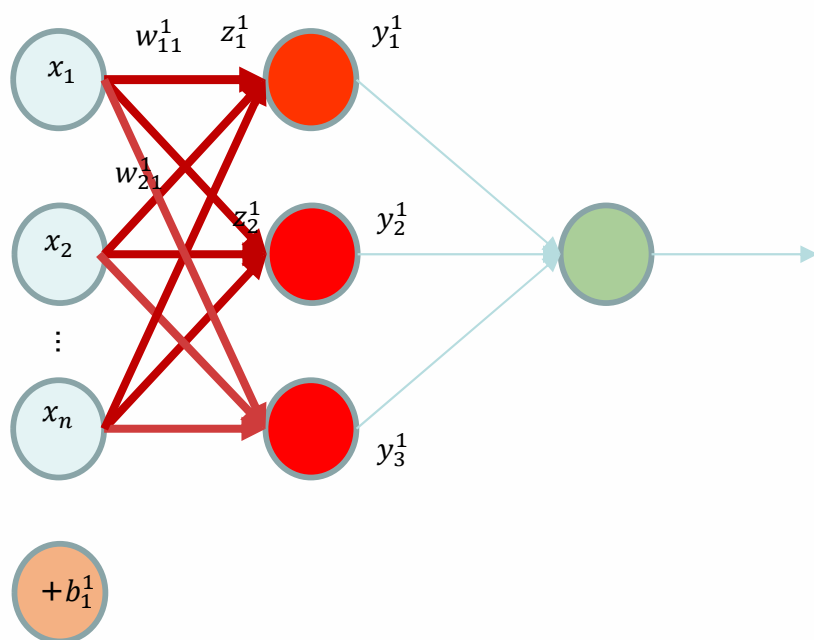
- $z_1^1 = w_{11}^1 x_1 + w_{21}^1 x_2 + \cdots + w_{n1}^1 x_n + b_1^1$
- $y_1^1 = f(z_1^1) = f(w_{11}^1 x_1 + w_{21}^1 x_2 + \cdots + w_{n1}^1 x_n + b_1^1)$

参数学习



信号正向传播

➤ 假定第 l 层神经元状态为 z^l ，经激活函数后的输出值为 y^l



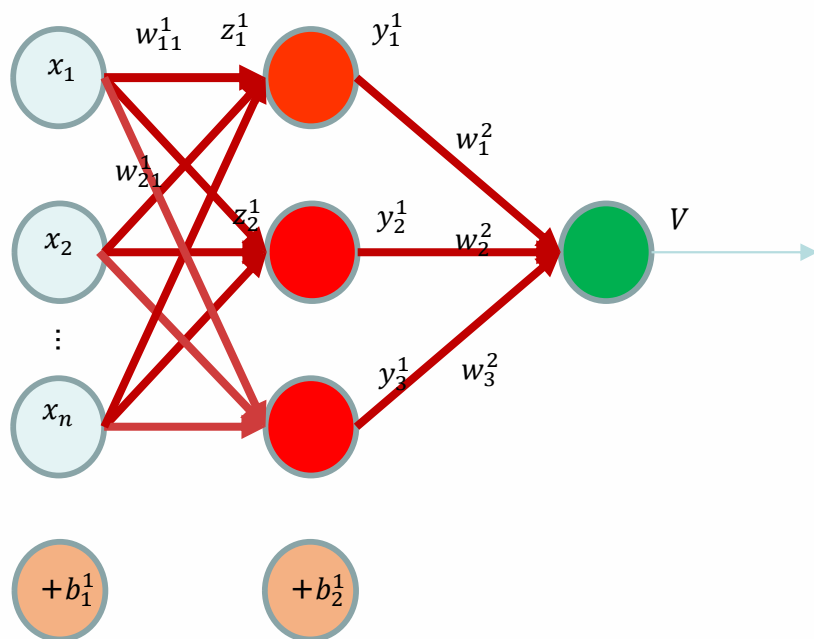
- $z_1^1 = w_{11}^1 x_1 + w_{21}^1 x_2 + \cdots + w_{n1}^1 x_n + b_1^1$
- $y_1^1 = f(z_1^1) = f(w_{11}^1 x_1 + w_{21}^1 x_2 + \cdots + w_{n1}^1 x_n + b_1^1)$
- $y_2^1 = f(z_2^1) = f(w_{12}^1 x_1 + w_{22}^1 x_2 + \cdots + w_{n2}^1 x_n + b_2^1)$
- $y_3^1 = f(z_3^1) = f(w_{13}^1 x_1 + w_{23}^1 x_2 + \cdots + w_{n3}^1 x_n + b_3^1)$

参数学习



信号正向传播

➤ 假定第 l 层神经元状态为 z^l ，经激活函数后的输出值为 y^l



$$V = f(w_1^2 y_1^1 + w_2^2 y_2^1 + \cdots + w_3^2 y_3^1 + b_2)$$

其中：

- $y_1^1 = f(z_1^1) = f(w_{11}^1 x_1 + w_{21}^1 x_2 + \cdots + w_{n1}^1 x_n + b_1^1)$
- $y_2^1 = f(z_2^1) = f(w_{12}^1 x_1 + w_{22}^1 x_2 + \cdots + w_{n2}^1 x_n + b_2^1)$
- $y_3^1 = f(z_3^1) = f(w_{13}^1 x_1 + w_{23}^1 x_2 + \cdots + w_{n3}^1 x_n + b_3^1)$

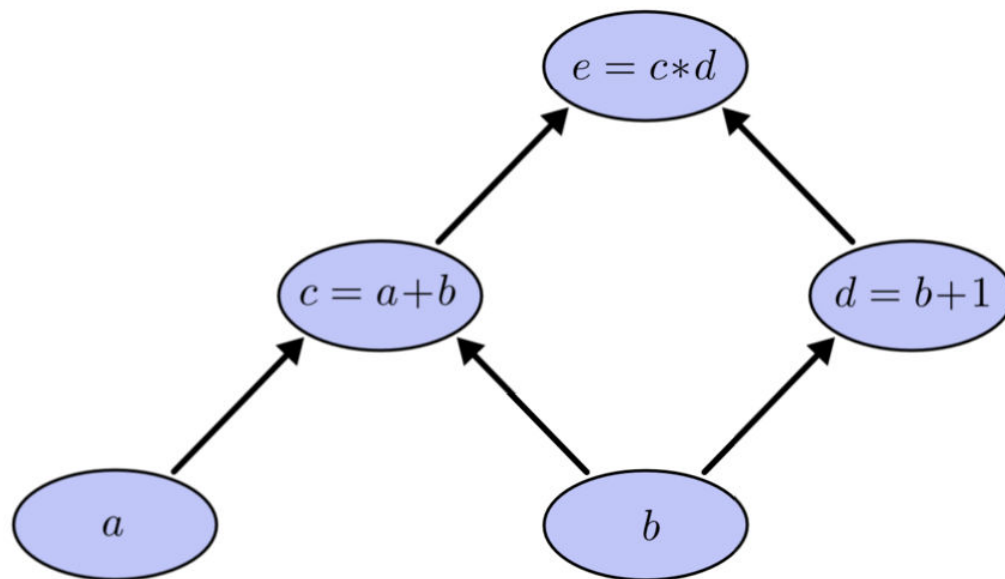
参数学习



反向传播算法

➤ 算法示例： $e = (a+b) * (b+1)$, 求 $\partial e / \partial a$, $\partial e / \partial b$

引入两个中间变量 c 和 d : $c = a + b$, $d = b + 1$, $e = c * d$



ref: <http://colah.github.io/posts/2015-08-Backprop/>

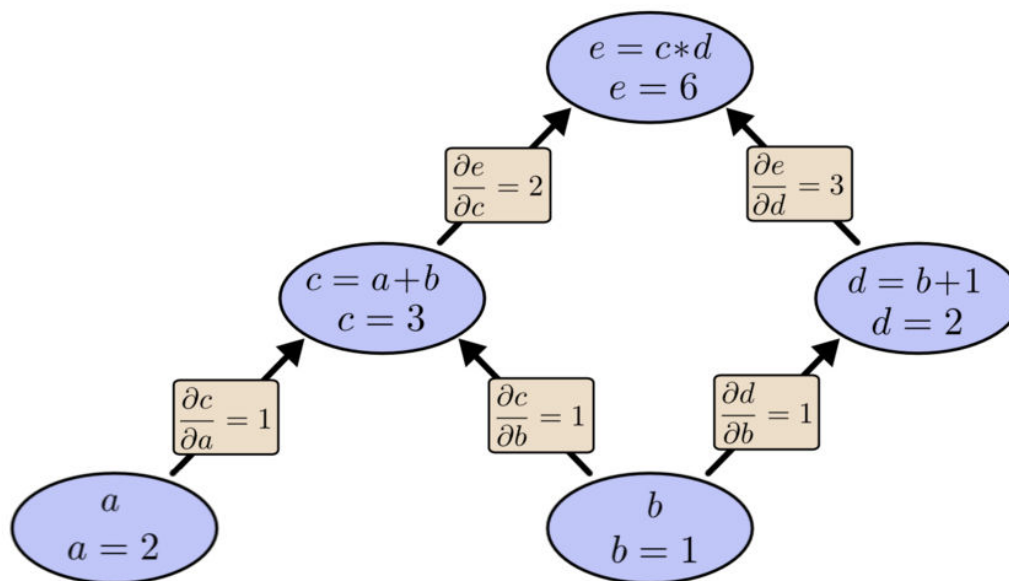
参数学习



反向传播算法

➤ 算法示例： $e = (a+b) * (b+1)$, 求 $\partial e / \partial a$, $\partial e / \partial b$

引入两个中间变量 c 和 d : $c = a+b$, $d = b+1$, $e = c*d$



ref: <http://colah.github.io/posts/2015-08-Backprop/>

参数学习

★ 反向传播算法

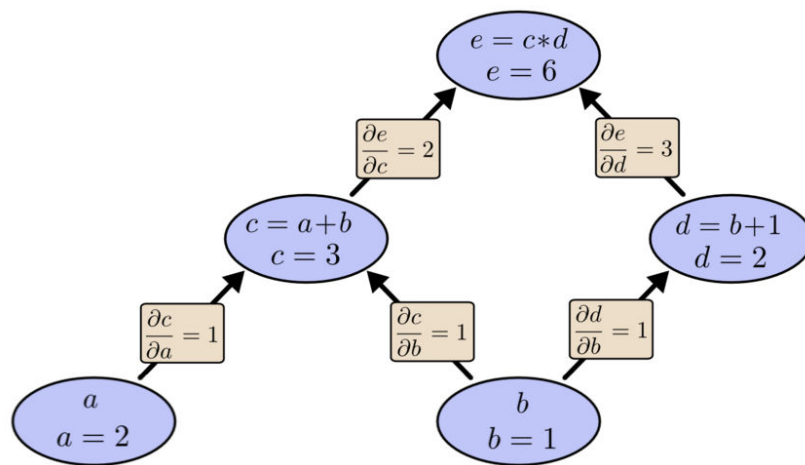
◆ 链式法则

➤ $\frac{\partial e}{\partial a} = \frac{\partial e}{\partial c} \cdot \frac{\partial c}{\partial a}$, 图中 $\frac{\partial e}{\partial a}$ 的值等于从a到e的路径上的偏导值的乘积

➤ $\frac{\partial e}{\partial b} = \frac{\partial e}{\partial c} \cdot \frac{\partial c}{\partial b} + \frac{\partial e}{\partial d} \cdot \frac{\partial d}{\partial b}$, 上图中 $\frac{\partial e}{\partial b}$ 的值等于从b到e的路径 (b-c-e)上的偏导值的乘积加上路径(b-d-e)上的偏导值的乘积。

➤ 若自下而上求解, 很多**路径被重复访问了**。

比如图中, 求 $\frac{\partial e}{\partial a}$ 需要计算路径a-c-e, 求 $\frac{\partial e}{\partial b}$ 都需要计算路径b-c-e和b-d-e, **路径c-e被访问了两次**。



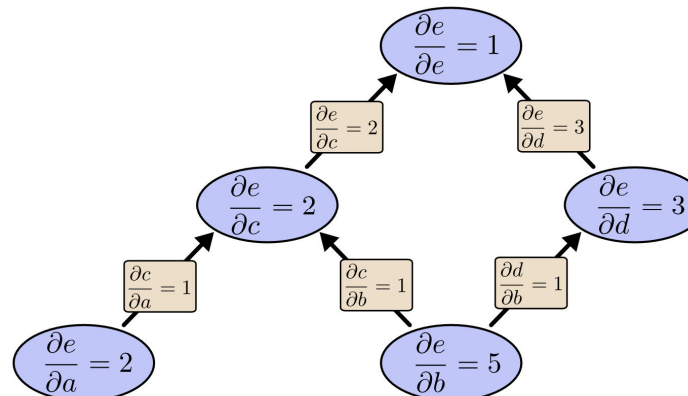
参数学习



反向传播算法

◆ 链式法则

自上而下：从最上层的节点e开始，对于e的下一层的所有子节点，将e的值（e是最顶点，值=1）乘以e到各个节点路径上的偏导值，并将结果发送到该子节点中。该子节点的值被设为“发送过来的值”，继续此过程向下传播



第一层：节点e初始值为1

第二层：节点e向节点c发送1*2，节点e向节点d发送1*3，节点c值为2. 节点d值为3.

第三层：节点c向a发送2*1，节点c向b发送2*1，节点d向b发送3*1
节点a值为2. 节点b值为为2*1+3*1=5.

即顶点e对a的偏导数为2 顶点e对b的偏导数为5

